

Network Edge Inference for Large Language Models: A Comprehensive Survey

Generated by SurveyForge

Contents

1	Introduction	3
2	Architectures and Models for Network Edge Inference	3
2.1	Introduction to Network Edge Inference Architectures	3
2.2	Mixture of Experts (MoE) Models for Edge Inference	4
2.3	Model Compression and Optimization Techniques	5
2.4	Edge-Cloud Collaboration and Distributed Inference	6
2.5	Security, Privacy, and Energy Efficiency Considerations	7
2.6	Applications and Future Directions	8
3	Optimization Techniques for Edge Inference	8
3.1	Quantization Techniques for Edge Inference	8
3.2	Sparse Coding and Low-Rank Approximations	9
3.3	Dynamic Voltage and Frequency Scaling for Edge Inference	10
3.4	Knowledge Distillation and Model Pruning for Efficient Edge Inference	11
3.5	Edge-Aware Inference Optimization	12
3.6	Emerging Trends and Future Directions in Edge Inference Optimization	13
4	Edge-Cloud Collaboration for Inference	14
4.1	Architectures for Edge-Cloud Collaborative Inference	14
4.2	Task Partitioning and Offloading Strategies	14
4.3	Communication Protocols and Data Management	15
4.4	Optimization Techniques for Edge-Cloud Inference	16
4.5	Security and Privacy Considerations	17
4.6	Applications and Use Cases	18
5	Security and Privacy Considerations	19
5.1	Threat Models and Vulnerability Analysis	19
5.2	Encryption and Access Control Mechanisms	19
5.3	Privacy-Preserving Inference Techniques	20
5.4	Edge-Cloud Collaboration for Secure Inference	21
5.5	Emerging Trends and Future Directions	22
6	Applications and Use Cases	22
6.1	Edge Computing Applications for Large Language Models	22
6.2	Natural Language Processing Tasks at the Edge	23
6.3	Enabling New Services and Applications with Edge Inference	25
6.4	Collaborative Edge-Cloud Inference for Large Language Models	25
6.5	Real-World Deployment and Case Studies	26
6.6	Future Directions and Emerging Trends	27

1 Introduction

The concept of Network Edge Inference for Large Language Models has emerged as a pivotal approach in reducing latency and enhancing real-time processing capabilities in various applications, including natural language processing, computer vision, and autonomous systems. As highlighted in [1], the ability to perform inference at the edge, closer to where data is generated, offers significant advantages over traditional cloud-based architectures. This is particularly important for applications requiring immediate responses, such as voice assistants, smart home devices, and real-time language translation.

One of the primary challenges in deploying Large Language Models (LLMs) at the edge is the substantial computational and memory requirements of these models. As noted in [2], training and inference of LLMs demand significant computational resources, making them difficult to deploy on edge devices with limited capabilities. To address this challenge, researchers have explored various techniques, including model pruning, quantization, and knowledge distillation, as discussed in [3]. These methods aim to reduce the computational footprint of LLMs while preserving their accuracy, thereby enabling their deployment on resource-constrained edge devices.

The significance of Network Edge Inference for LLMs is further underscored by its potential to improve privacy and security. By performing inference at the edge, sensitive user data can be processed locally without being transmitted to the cloud, reducing the risk of data breaches and unauthorized access. As emphasized in [4], this approach also offers energy efficiency benefits, as it minimizes the need for data transmission and reduces the carbon footprint associated with cloud computing.

Recent studies, such as [5] and [6], have explored the application of LLMs in edge computing scenarios, including smart homes, autonomous vehicles, and healthcare devices. These applications demonstrate the potential of Network Edge Inference to enable real-time, personalized, and secure services that can transform various aspects of our lives.

Despite the promising advancements in Network Edge Inference for LLMs, several challenges and opportunities remain to be addressed. As noted in [7], the development of efficient inference algorithms, model compression techniques, and edge-cloud collaboration architectures is crucial for widespread adoption. Furthermore, ensuring the security and privacy of user data, as well as addressing the ethical implications of LLMs, are essential for building trust and confidence in these technologies.

In conclusion, Network Edge Inference for Large Language Models represents a critical area of research and development, with significant implications for various applications and industries. As highlighted in [8] and [9], the ability to perform efficient and secure inference at the edge will be essential for unlocking the full potential of LLMs. By addressing the challenges and opportunities in this field, researchers and practitioners can contribute to the development of more efficient, secure, and privacy-preserving AI systems that can transform our lives and societies. Future research directions, as discussed in [10] and [11], should focus on developing innovative solutions that balance performance, efficiency, and security, ultimately enabling the widespread adoption of Network Edge Inference for LLMs.

2 Architectures and Models for Network Edge Inference

2.1 Introduction to Network Edge Inference Architectures

The proliferation of Large Language Models (LLMs) has revolutionized the landscape of natural language processing, offering unprecedented capabilities in text generation, comprehension, and analysis. However, the deployment of these models in real-world applications is hindered by their substantial computational and memory requirements, which can lead to significant latency and energy consumption. To address these challenges, Network Edge Inference has emerged as a

promising paradigm, enabling the efficient and real-time processing of LLMs at the network edge. This subsection delves into the fundamental concepts and architectures designed for Network Edge Inference, highlighting their role in facilitating the widespread adoption of LLMs in edge computing scenarios.

At the heart of Network Edge Inference lies the concept of edge computing, which involves processing data closer to its source, thereby reducing latency and improving real-time processing capabilities [5]. Edge computing architectures, such as fog computing and hierarchical edge-cloud models, have been proposed to support the deployment of LLMs at the network edge [12]. These architectures leverage the strengths of both edge and cloud computing, enabling the efficient allocation of computational resources and adaptive handling of varying workloads. For instance, the EdgePipe framework [13] utilizes pipeline parallelism to speed up inference and enable the deployment of larger models that would otherwise be constrained by edge device resources.

Model compression techniques, such as pruning, quantization, and knowledge distillation, have also been extensively explored to reduce the computational footprint of LLMs [3]. These techniques enable the deployment of LLMs on resource-constrained edge devices, while maintaining acceptable levels of accuracy. For example, the EdgeBERT model [14] achieves significant energy reductions through sentence-level energy optimizations, making it suitable for deployment on edge devices. Furthermore, the use of Mixture of Experts (MoE) models [15] has been proposed as a means of improving the efficiency of LLM inference at the edge, by activating only a subset of the model’s parameters during computation.

The design of efficient LLM inference systems for edge devices is a complex task, requiring careful consideration of factors such as model architecture, computational resources, and energy consumption [16]. Recent studies have investigated the use of FPGA-based accelerators [16] and GPU-based architectures [17] to improve the efficiency of LLM inference on edge devices. Additionally, the development of novel inference algorithms, such as the Lookahead framework [18], has been proposed to accelerate LLM inference on edge devices.

In conclusion, Network Edge Inference offers a promising solution for the efficient and real-time processing of LLMs at the network edge. Through the use of edge computing architectures, model compression techniques, and efficient inference algorithms, it is possible to deploy LLMs on resource-constrained edge devices, while maintaining acceptable levels of accuracy. As the field continues to evolve, future research directions may include the development of more efficient model architectures, the exploration of novel inference algorithms, and the investigation of emerging trends such as the use of quantum computing [19] and edge AI [6] for LLM inference. By addressing the challenges associated with LLM deployment at the network edge, researchers can unlock the full potential of these models and enable their widespread adoption in a range of applications, from smart homes and autonomous vehicles to healthcare and education [1].

2.2 Mixture of Experts (MoE) Models for Edge Inference

The Mixture of Experts (MoE) model architecture has garnered significant attention in recent years due to its potential to enhance the efficiency and performance of large language models (LLMs) in edge inference scenarios. As highlighted in [20], MoE models achieve this by sparsely activating only a subset of the model’s parameters during computation, thereby reducing computational costs. Building on the concepts discussed earlier, this subsection delves into the specifics of MoE models, their architecture, and their application in Network Edge Inference, including recent advancements and techniques to improve their efficiency and performance.

At its core, a MoE model consists of multiple expert networks, each specializing in a specific subset of the input data, and a gating network that determines which experts to activate for a given input. This architecture allows MoE models to capture complex patterns and relationships in the data while minimizing computational overhead. The effectiveness of MoE models in achieving state-of-the-art results on various natural language processing tasks while reducing the computational requirements is demonstrated in [21]. The use of MoE models in edge inference

scenarios is particularly promising, as it enables the deployment of large language models on resource-constrained devices while maintaining performance, which is a crucial aspect of Network Edge Inference.

Recent advancements in MoE models have focused on improving their efficiency and performance. For instance, [22] introduces a novel MoE architecture that achieves significant performance gains while reducing computational costs. The authors propose a sparse MoE architecture that adaptively selects the most relevant experts for each input, resulting in substantial reductions in computational overhead. Similarly, [23] presents an open-source MoE model that achieves state-of-the-art results on various benchmarks while providing a highly efficient and scalable architecture. These advancements have paved the way for the widespread adoption of MoE models in edge inference scenarios.

Techniques such as hierarchical MoE [24] and low-rank adaptations [25] have also been proposed to further improve the efficiency and performance of MoE models. These techniques enable the deployment of MoE models on resource-constrained devices while maintaining performance, making them particularly suitable for edge inference scenarios. Moreover, [15] introduces an adaptive expert gating mechanism that dynamically adjusts the number of activated experts based on the input data, resulting in significant reductions in computational overhead. The integration of these techniques with other model compression and optimization methods, such as quantization and pruning, can lead to further improvements in efficiency and performance, as discussed in the following section.

The application of MoE models in edge inference scenarios has also been explored in various studies. For instance, [26] proposes a personalized inference scheduling framework that leverages edge-cloud collaboration to optimize the deployment of MoE models on resource-constrained devices. The authors demonstrate significant performance gains and reductions in computational overhead, making MoE models a promising solution for edge inference scenarios. Similarly, [27] presents a collaborative edge computing framework that enables the efficient deployment of MoE models on resource-constrained devices, resulting in significant performance gains and reductions in computational overhead. These studies highlight the potential of MoE models to enable the deployment of large language models on resource-constrained devices while maintaining performance.

In conclusion, MoE models have emerged as a promising solution for edge inference scenarios, offering significant performance gains and reductions in computational overhead. Recent advancements in MoE models, such as sparse MoE architectures, hierarchical MoE, and low-rank adaptations, have further improved their efficiency and performance. The application of MoE models in edge inference scenarios has also been explored in various studies, demonstrating their potential to enable the deployment of large language models on resource-constrained devices while maintaining performance. As the field continues to evolve, future research directions may focus on further improving the efficiency and performance of MoE models, exploring new applications and scenarios, and addressing the challenges associated with their deployment on resource-constrained devices, ultimately unlocking the full potential of Network Edge Inference.

2.3 Model Compression and Optimization Techniques

Model compression and optimization techniques are essential for efficient Network Edge Inference, as they enable the deployment of large language models on resource-constrained edge devices. One of the key techniques in this domain is quantization, which involves reducing the precision of model weights and activations to decrease memory usage and improve inference speed. Quantization methods have been shown to achieve significant reductions in model size while maintaining accuracy. However, quantization can also lead to accuracy loss, and techniques such as quantization-aware training are necessary to mitigate this issue.

Another crucial technique for model compression is pruning, which involves eliminating redundant or unnecessary weights and connections in the model [28]. Pruning can be performed

using various methods, including unstructured, structured, and iterative pruning, each with its strengths and weaknesses. Knowledge distillation is also a popular technique for model compression, where a smaller model (the student) is trained to mimic the behavior of a larger model (the teacher). This approach has been shown to achieve significant reductions in model size while maintaining performance.

Early exit strategies are another technique used to optimize model inference, where the model is designed to exit early for certain inputs, reducing computational costs [29]. These strategies can be particularly effective in edge computing scenarios, where resources are limited and latency is critical. In addition to these techniques, sparse mixture-of-experts (MoE) models have gained significant attention in recent years, as they offer a promising approach to scaling large language models while maintaining efficiency [30]. MoE models activate only a subset of experts for each input, reducing computational costs and enabling deployment on edge devices.

The combination of these techniques can lead to significant improvements in model efficiency and performance. For example, the use of quantization and pruning can be combined with knowledge distillation to achieve state-of-the-art results. Furthermore, the integration of MoE models with early exit strategies and other optimization techniques can enable the deployment of large language models on edge devices while maintaining performance and efficiency [31].

In conclusion, model compression and optimization techniques are crucial for efficient Network Edge Inference, and techniques such as quantization, pruning, knowledge distillation, and early exit strategies have been shown to achieve significant improvements in model efficiency and performance. The combination of these techniques, particularly with MoE models, offers a promising approach to scaling large language models while maintaining efficiency. As the demand for edge AI continues to grow, the development of efficient and optimized models will be critical to enabling the deployment of large language models on resource-constrained edge devices.

2.4 Edge-Cloud Collaboration and Distributed Inference

The collaboration between edge and cloud computing for Large Language Model (LLM) inference has gained significant attention in recent years, as it offers a promising solution to balance latency and computational resources [32]. Building on the concepts of model compression and optimization techniques discussed earlier, edge-cloud collaboration enables efficient distributed inference, reducing the computational burden on edge devices while minimizing latency [12]. This collaborative approach is crucial for deploying large language models on resource-constrained edge devices, as it allows for the allocation of tasks to the most suitable computing resources, thereby improving overall efficiency.

One key aspect of edge-cloud collaboration is the design of architectures that can efficiently partition tasks between edge and cloud computing [33]. For instance, fog computing and hierarchical edge-cloud models have been explored as potential architectures for edge-cloud collaborative inference [34]. These architectures aim to reduce latency and improve computational efficiency by allocating tasks to the most suitable computing resources [35]. Furthermore, task partitioning strategies, such as computation offloading and data offloading, play a crucial role in optimizing resource utilization and minimizing latency [36].

In addition to architecture design and task partitioning, communication protocols and data management strategies are also essential for efficient edge-cloud collaboration [37]. For example, low-latency communication protocols and data synchronization techniques can ensure seamless collaboration between edge and cloud computing [38]. Additionally, consistency models and security considerations are critical to ensuring the reliability and trustworthiness of edge-cloud collaborative inference [3].

Recent studies have demonstrated the effectiveness of edge-cloud collaboration in improving the efficiency of LLM inference [39]. For instance, [40] proposed a compression scheme that combines block removal and block parameter sharing to reduce model size and latency. Similarly, [41] introduced a neural architecture search framework that optimizes models for efficient

inference on edge devices. These studies highlight the potential of edge-cloud collaboration in enabling efficient and scalable LLM inference [42].

However, edge-cloud collaboration for LLM inference also poses several challenges and open research questions [43]. For example, the optimal partitioning of tasks between edge and cloud computing remains an open problem [44]. Moreover, the development of efficient communication protocols and data management strategies that can handle the unique requirements of LLM inference is an active area of research [45]. As the field continues to evolve, addressing these challenges will be crucial for unlocking the full potential of edge-cloud collaboration for LLM inference.

In conclusion, edge-cloud collaboration offers a promising solution for efficient LLM inference, enabling the balancing of latency and computational resources [46]. As researchers continue to explore this area, it is essential to consider the security, privacy, and energy efficiency implications of edge-cloud collaboration, which will be discussed in the following section [47]. By developing innovative architectures, task partitioning strategies, and communication protocols that can efficiently facilitate edge-cloud collaboration, researchers can enable efficient, scalable, and reliable deployment of large language models in real-world applications [48]. Future studies should focus on addressing the challenges and open research questions associated with edge-cloud collaboration, ultimately leading to more efficient and scalable LLM inference [49].

2.5 Security, Privacy, and Energy Efficiency Considerations

The security, privacy, and energy efficiency of Network Edge Inference are critical considerations, as they directly impact the reliability, trustworthiness, and sustainability of edge-based large language models [47]. Threat models for edge inference systems include data breaches, model inversion attacks, and denial-of-service attacks, which can compromise user data and model integrity [50]. To mitigate these threats, encryption mechanisms such as homomorphic encryption and secure multi-party computation can be employed to protect user data and models [51]. Additionally, access control mechanisms and differential privacy techniques can be used to regulate access to models and data, ensuring that sensitive information is not compromised [52].

Energy efficiency is another crucial aspect of Network Edge Inference, as edge devices are often resource-constrained and power-limited [53]. Power management strategies such as dynamic voltage and frequency scaling, power-aware scheduling, and energy harvesting can be used to minimize energy consumption while maintaining inference performance [17]. Furthermore, model compression techniques such as quantization, pruning, and knowledge distillation can be applied to reduce the computational footprint of large language models, resulting in significant energy savings [54].

The trade-offs between security, privacy, and energy efficiency must be carefully evaluated, as enhancing one aspect may compromise another [55]. For instance, implementing robust encryption mechanisms may increase energy consumption, while model compression techniques may compromise model accuracy [56]. Therefore, a balanced approach that considers the specific requirements and constraints of the edge environment is essential [57].

Emerging trends and challenges in Network Edge Inference include the integration of new technologies such as 5G networks, edge AI chips, and neuromorphic computing, which can enhance security, privacy, and energy efficiency [58]. Additionally, the development of more efficient models, such as Mixture of Experts (MoE) models, can reduce energy consumption while maintaining inference performance [15]. However, these advancements also introduce new challenges, such as ensuring the security and privacy of user data in distributed edge environments [59].

In conclusion, the security, privacy, and energy efficiency of Network Edge Inference are complex and interconnected considerations that require careful evaluation and balancing [60]. By leveraging encryption mechanisms, access control policies, power management strategies, and model compression techniques, edge-based large language models can be deployed securely,

privately, and sustainably [61]. As the field continues to evolve, emerging trends and challenges must be addressed through innovative solutions and architectures that prioritize security, privacy, and energy efficiency [62].

2.6 Applications and Future Directions

The deployment of Large Language Models (LLMs) at the network edge has numerous applications across various domains, including smart homes, autonomous vehicles, healthcare, and education, which can significantly benefit from the security, privacy, and energy efficiency considerations discussed earlier. In smart homes, for instance, edge-based LLMs can enhance voice assistants' capabilities, enabling more efficient and private control over home automation systems [14]. Similarly, in autonomous vehicles, LLMs can facilitate natural language-based navigation and decision-making, improving safety and user experience [63].

The use of LLMs in healthcare can also significantly benefit from edge inference, particularly in applications such as medical chatbots and personalized patient care, where privacy and real-time processing are crucial [64]. Furthermore, edge-based LLMs can support educational platforms by providing personalized learning experiences and real-time feedback, leveraging the advantages of localized processing and reduced latency [65]. These applications highlight the need for a balanced approach to security, privacy, and energy efficiency, as discussed in the previous section, to ensure the reliable and trustworthy deployment of LLMs at the network edge.

Future research directions in Network Edge Inference for LLMs include the development of more efficient models, improved edge-cloud collaboration architectures, and the integration of emerging technologies such as 5G and 6G networks [58]. Additionally, there is a growing need for sustainable and energy-efficient edge inference solutions, which can be achieved through innovative approaches like model quantization, pruning, and knowledge distillation [66]. The collaboration between edge and cloud computing is also expected to play a critical role in the future of LLM inference, offering a balance between latency and computational resources [12].

Techniques such as distributed inference, where parts of the model are processed at the edge and others in the cloud, can significantly enhance the efficiency and scalability of LLM deployments [67]. Moreover, the application of LLMs in edge environments raises important considerations regarding security and privacy, particularly in scenarios where sensitive data is involved [50]. Addressing these challenges through the development of secure and privacy-preserving inference protocols will be essential for the widespread adoption of edge-based LLMs [68]. By exploring new architectures, models, and techniques, the academic community can contribute to the development of more sustainable, efficient, and secure edge AI solutions, ultimately unlocking the full potential of LLMs in real-world scenarios [60], which will be further discussed in the following section.

3 Optimization Techniques for Edge Inference

3.1 Quantization Techniques for Edge Inference

Quantization techniques have emerged as a crucial strategy for optimizing the performance of Large Language Models (LLMs) at the network edge, where resources are limited and efficiency is paramount. By reducing the precision of model weights and activations, quantization enables significant reductions in memory footprint and computational overhead, thereby improving inference speed [69]. This subsection delves into the various quantization methods employed for edge inference, including weight quantization, activation quantization, and quantization-aware training, to provide a comprehensive understanding of their strengths, limitations, and applications.

Weight quantization is a widely adopted technique that involves reducing the precision of model weights from 32-bit floating-point numbers to lower-bit integers, such as 8-bit or 16-bit integers [1]. This reduction in precision leads to a significant decrease in memory usage, making it possible to deploy larger models on edge devices with limited memory capacity. However, weight quantization can result in a loss of model accuracy, particularly if the quantization scheme is not carefully designed [70]. To mitigate this issue, techniques such as quantization-aware training have been proposed, which involve training the model with simulated quantization noise to improve its robustness to quantization errors [8].

Activation quantization is another important technique used in edge inference, which involves reducing the precision of activations during inference [4]. By quantizing activations, the computational overhead of inference can be significantly reduced, leading to improved latency and energy efficiency. However, activation quantization can be more challenging than weight quantization, as activations can have a large range of values and may require more sophisticated quantization schemes to maintain model accuracy [35]. Recent studies have explored the use of advanced quantization schemes, such as logarithmic quantization and adaptive quantization, to improve the accuracy and efficiency of activation quantization [10].

Quantization-aware training is a critical component of quantization techniques, as it enables the model to adapt to the quantization errors introduced during inference [11]. By simulating quantization noise during training, the model can learn to compensate for the errors introduced by quantization, leading to improved accuracy and robustness. Quantization-aware training can be applied to both weight and activation quantization, and can be used in conjunction with other optimization techniques, such as pruning and knowledge distillation, to further improve model efficiency [3].

In addition to these techniques, recent studies have explored the use of emerging trends, such as post-training quantization and integer-only quantization, to further improve the efficiency of LLMs at the edge [66]. Post-training quantization involves quantizing the model after training, without requiring any modifications to the training process, while integer-only quantization involves representing both weights and activations using integers, eliminating the need for floating-point operations during inference [71].

In conclusion, quantization techniques play a vital role in optimizing the performance of LLMs at the network edge, enabling significant reductions in memory footprint and computational overhead. By carefully designing and applying weight quantization, activation quantization, and quantization-aware training, it is possible to achieve improved inference speed and efficiency, while maintaining model accuracy. As the field continues to evolve, emerging trends and techniques, such as post-training quantization and integer-only quantization, are likely to further improve the efficiency and effectiveness of LLMs at the edge, enabling a wide range of applications and use cases [5]. Future research directions may include exploring the use of advanced quantization schemes, such as adaptive and logarithmic quantization, and developing more sophisticated quantization-aware training methods to further improve model accuracy and robustness [6].

3.2 Sparse Coding and Low-Rank Approximations

The increasing complexity and size of Large Language Models (LLMs) have led to significant challenges in their deployment, particularly at the edge, where computational resources and memory are limited. As discussed earlier, quantization techniques have emerged as a crucial strategy for optimizing the performance of LLMs at the network edge. However, to further improve the efficiency and accuracy of LLMs, sparse coding techniques and low-rank approximations have been proposed as promising approaches for efficient representation and computation in LLMs [72]. Sparse coding techniques aim to reduce the memory footprint of LLMs by eliminating redundant parameters, while low-rank approximations aim to reduce the dimensionality of the model’s weight matrices by representing them as a product of two smaller matrices, thereby reducing the number of parameters and computations required [73].

One of the key benefits of sparse coding is its ability to reduce the memory footprint of LLMs, making them more suitable for deployment on edge devices with limited memory capacity [74]. For instance, [75] demonstrated that sparse coding can be used to prune LLMs to at least 50% sparsity in one-shot, without any retraining, resulting in significant reductions in memory usage and computational overhead. Similarly, low-rank approximations have been shown to be effective in reducing the computational overhead of LLMs, with [76] demonstrating that low-rank approximations can be used to reduce the number of parameters in LLMs by up to 25%, resulting in significant improvements in inference speed.

Despite the benefits of sparse coding and low-rank approximations, there are several challenges associated with their application to LLMs. One of the key challenges is the need to balance the trade-off between model accuracy and computational overhead [77]. For instance, [78] demonstrated that simple depth pruning can effectively compress LLMs while achieving comparable or superior performance to recent width pruning studies. However, the choice of pruning strategy and the degree of sparsity or low-rank approximation can significantly impact the model’s accuracy, making it challenging to determine the optimal approach for a given application.

Recent studies have explored the use of hybrid approaches that combine sparse coding and low-rank approximations to achieve further improvements in efficiency [25]. For example, [79] introduced an adaptive pruning and tuning approach that combines sparse coding and low-rank approximations to improve the efficiency of LLMs. The approach involves dynamically adding salient tuning parameters for fast and accurate convergence while discarding unimportant parameters for efficiency. Similarly, [80] proposed a hardware-aware pruning framework that incorporates genuine inference speed-up sensitivity from each pruning structure, allowing for cross-layer optimization and compilation optimization during pruning.

The application of sparse coding and low-rank approximations to LLMs has significant implications for edge AI, enabling the deployment of more efficient and accurate models on resource-constrained devices [65]. For instance, [54] demonstrated that small language models can be trained and deployed on edge devices, achieving high token rates and accuracy while reducing memory usage and computational overhead. Similarly, [27] proposed a collaborative edge computing framework that leverages model partitioning and device selection to achieve efficient LLM inference on edge devices. These approaches can be further optimized using dynamic voltage and frequency scaling (DVFS) techniques, which will be discussed in the next section.

In conclusion, sparse coding and low-rank approximations offer promising approaches for efficient representation and computation in LLMs, enabling their deployment on edge devices with limited computational resources and memory. While there are several challenges associated with their application, recent studies have explored hybrid approaches that combine sparse coding and low-rank approximations to achieve further improvements in efficiency. As the field continues to evolve, it is likely that we will see the development of more sophisticated techniques for sparse coding and low-rank approximations, enabling the widespread adoption of LLMs in edge AI applications [81]. Future research directions may include the exploration of novel pruning strategies, the development of more efficient algorithms for sparse coding and low-rank approximations, and the application of these techniques to other areas of AI research, such as computer vision and robotics [15].

3.3 Dynamic Voltage and Frequency Scaling for Edge Inference

Dynamic voltage and frequency scaling (DVFS) is a crucial technique for optimizing energy efficiency during Large Language Model inference at the network edge. By adjusting the voltage and frequency of processing units, DVFS balances performance and power consumption, making it an essential approach for edge computing [63]. The primary goal of DVFS is to minimize energy consumption while meeting the required latency and throughput constraints. This is

particularly important in edge computing, where devices often have limited power resources and stringent latency requirements [82].

Various DVFS strategies have been proposed to optimize energy efficiency in edge computing. One approach is to use a feedback control system that monitors the system’s performance and adjusts the voltage and frequency accordingly [83]. Another approach is to use machine learning algorithms to predict the optimal voltage and frequency settings based on the workload and system characteristics [56]. For example, the [63] paper presents a DVFS strategy that uses a combination of feedback control and machine learning to optimize energy efficiency in edge computing.

In addition to these strategies, several papers have explored the use of DVFS in specific edge computing applications, such as language modeling [22] and computer vision [84]. These papers demonstrate the effectiveness of DVFS in reducing energy consumption while maintaining performance in these applications. For instance, the [22] paper shows that DVFS can reduce energy consumption by up to 30% in language modeling tasks.

However, DVFS also has its limitations and challenges. One of the main challenges is the complexity of implementing DVFS in edge computing systems, which often have limited resources and stringent latency requirements [60]. Another challenge is the need to balance energy efficiency with performance, as reducing voltage and frequency can impact the system’s throughput and latency [85]. Furthermore, DVFS can also introduce additional overhead, such as the energy consumed by the voltage regulator and the time spent on voltage and frequency transitions [86].

Despite these challenges, DVFS remains a promising technique for optimizing energy efficiency in edge computing. Future research directions include exploring new DVFS strategies that can better balance energy efficiency and performance, as well as developing more efficient and scalable DVFS implementations [87]. Additionally, there is a need to investigate the use of DVFS in emerging edge computing applications, such as autonomous vehicles and smart cities [88]. By addressing these challenges and exploring new research directions, DVFS can play a critical role in enabling efficient and sustainable edge computing.

The use of DVFS in edge computing also raises several interesting research questions. For example, how can DVFS be used to optimize energy efficiency in edge computing systems with multiple processing units [31]? How can DVFS be integrated with other energy-efficient techniques, such as dynamic power management and voltage scaling [89]? Answering these questions will require further research and experimentation, but the potential benefits of DVFS in edge computing make it an exciting and worthwhile area of study.

In conclusion, DVFS is a powerful technique for optimizing energy efficiency in edge computing, and its application in Large Language Model inference is particularly promising [30]. By adjusting the voltage and frequency of processing units, DVFS can balance performance and power consumption, making it an essential approach for edge computing. While there are challenges and limitations to using DVFS in edge computing, the potential benefits make it an exciting and worthwhile area of research [90]. As edge computing continues to evolve and grow, the use of DVFS will play a critical role in enabling efficient and sustainable computing at the edge.

3.4 Knowledge Distillation and Model Pruning for Efficient Edge Inference

The pursuit of efficient edge inference for large language models has led to the development of various optimization techniques, among which knowledge distillation and model pruning have emerged as particularly effective methods. Knowledge distillation involves transferring the knowledge from a large, pre-trained model (the teacher) to a smaller model (the student), enabling the smaller model to achieve comparable performance to the larger one. This technique is especially useful in the context of edge inference, where computational resources and memory are limited, and techniques like dynamic voltage and frequency scaling (DVFS) can be used to

further optimize energy efficiency. By distilling the knowledge from a large language model into a smaller one, the resulting model can be deployed on edge devices with reduced latency and improved real-time processing capabilities, which is crucial for applications that require fast and accurate language processing.

Model pruning, on the other hand, involves eliminating redundant or unnecessary weights and connections within a neural network to reduce its computational footprint [91]. This technique can be applied to large language models to create more efficient versions that require less computational power and memory, making them more suitable for edge inference. Structured pruning methods, such as [72], have been shown to be effective in reducing the size of large language models while preserving their performance. Unstructured pruning methods, such as [75], have also been proposed, which can achieve high sparsity levels in large language models without requiring retraining. The combination of knowledge distillation and model pruning can lead to even more efficient models, enabling the deployment of large language models on resource-constrained edge devices.

Recent studies have also explored the use of mixture-of-experts (MoE) models for efficient edge inference [21]. MoE models consist of multiple expert models, each of which is responsible for a specific subset of the input data. By activating only a subset of the expert models for each input, MoE models can achieve significant reductions in computational cost while maintaining performance. [32] proposes an adaptive model splitting algorithm that can be used to optimize the placement of MoE models on edge devices. Additionally, other methods such as quantization [92] and low-rank approximation [93] have been proposed to reduce the computational footprint of large language models.

The development of efficient edge inference techniques for large language models is an active area of research, with many open challenges and opportunities for innovation. As the demand for real-time language processing on edge devices continues to grow, the need for efficient and effective optimization techniques will become increasingly important. Future research directions may include the development of new knowledge distillation and model pruning techniques, as well as the exploration of novel architectures and algorithms that can efficiently process language inputs on edge devices [3]. Furthermore, the integration of edge inference with other emerging technologies, such as 5G networks and the Internet of Things (IoT), may lead to new opportunities for efficient language processing in a wide range of applications [41]. By building on these optimization techniques and exploring new approaches, we can enable the efficient deployment of large language models on edge devices, paving the way for a wide range of new applications and services that can transform the way we live and work.

3.5 Edge-Aware Inference Optimization

The optimization of Large Language Model (LLM) inference at the network edge is a complex task that requires careful consideration of the unique constraints and opportunities presented by edge environments. Edge-aware inference optimization techniques aim to improve the efficiency and effectiveness of LLM inference by leveraging the characteristics of edge devices and networks. One key approach is to design models that are aware of the edge environment and can adapt to the available resources [14]. This can involve techniques such as dynamic voltage and frequency scaling, which can reduce energy consumption while maintaining performance.

Another important consideration is the use of collaborative inference techniques, which can distribute the computation of LLM inference across multiple edge devices [67]. This can help to reduce the computational load on individual devices and improve overall system performance. However, collaborative inference also introduces new challenges, such as the need for efficient communication protocols and data management strategies [94]. To address these challenges, researchers have proposed various techniques, including the use of over-the-air computation [94] and adaptive layer splitting [95].

In addition to these technical challenges, edge-aware inference optimization must also consider

the practical implications of deploying LLMs in real-world edge environments. This includes issues such as privacy, security, and explainability [52]. To address these concerns, researchers have proposed various techniques, including the use of federated learning [96] and differential privacy [97].

Despite the challenges and complexities of edge-aware inference optimization, the potential benefits of this approach are significant. By improving the efficiency and effectiveness of LLM inference at the network edge, we can enable a wide range of new applications and services, from smart homes and cities to autonomous vehicles and healthcare [5]. To realize these benefits, however, further research is needed to develop and refine edge-aware inference optimization techniques. This includes the development of new models and algorithms that are specifically designed for edge environments [54], as well as the creation of new hardware and software platforms that can support the efficient and effective deployment of LLMs at the edge [56].

In conclusion, edge-aware inference optimization is a critical area of research that has the potential to unlock the full potential of Large Language Models in edge environments. By developing and refining techniques that are specifically designed for edge environments, we can improve the efficiency and effectiveness of LLM inference and enable a wide range of new applications and services. Further research is needed to address the technical challenges and practical implications of edge-aware inference optimization, but the potential benefits of this approach make it an exciting and important area of study [60]. As noted in [98], the integration of large and small language models can lead to improved performance and efficiency, and [26] demonstrates the importance of personalized inference scheduling for diverse LLM services. Overall, the development of edge-aware inference optimization techniques has the potential to revolutionize the way we deploy and use Large Language Models in edge environments.

3.6 Emerging Trends and Future Directions in Edge Inference Optimization

The optimization of Large Language Model (LLM) inference at the edge is a rapidly evolving field, driven by the need for efficient, scalable, and secure processing of complex natural language tasks in real-time applications. As discussed earlier, edge-aware inference optimization techniques have been developed to improve the efficiency and effectiveness of LLM inference by leveraging the characteristics of edge devices and networks. Building on these advancements, recent developments in edge computing, model compression, and distributed inference have paved the way for the deployment of LLMs in resource-constrained environments, enabling new applications and services that were previously unimaginable [14].

One of the key trends shaping the landscape of edge inference optimization is the integration of new technologies, such as 5G networks, edge AI chips, and neuromorphic computing, which are expected to play a crucial role in enabling more efficient and scalable edge inference [6]. For instance, the use of 5G networks can provide ultra-low latency and high-bandwidth connectivity, enabling the seamless transmission of data between edge devices and the cloud [58]. Additionally, edge AI chips, such as those developed by companies like Google and Apple, are designed to provide high-performance processing while minimizing power consumption, making them ideal for edge inference applications [16].

Another emerging trend is the development of novel model compression techniques, such as quantization, pruning, and knowledge distillation, which can significantly reduce the computational footprint of LLMs while maintaining their accuracy [64]. These techniques have been shown to be effective in reducing the memory requirements and computational overhead of LLMs, making them more suitable for deployment on edge devices [80]. Furthermore, the use of distributed inference techniques, such as split inference and pipeline parallelism, can also help to reduce the computational overhead of LLMs and improve their scalability [99].

The increasing focus on security and privacy is also expected to drive the development of new techniques and technologies for edge inference optimization. For instance, the use of secure multi-party computation and homomorphic encryption can enable the secure processing

of sensitive data on edge devices [50]. Additionally, the development of privacy-preserving techniques, such as differential privacy and federated learning, can help to protect user data and maintain the confidentiality of edge inference applications [51].

As the field of edge inference optimization continues to evolve, future research directions are expected to focus on the development of autonomous edge inference systems that can adapt to changing environmental conditions and optimize their performance in real-time [95]. Such systems can leverage techniques like reinforcement learning and meta-learning to optimize their performance and improve their scalability [26]. Another area of research that holds significant promise is the development of edge inference systems that can leverage the capabilities of multiple edge devices and cloud servers to provide scalable and secure processing of LLMs [27]. These advancements will be crucial in enabling the widespread adoption of LLMs in edge environments and unlocking their full potential to transform the way we live and work [65].

4 Edge-Cloud Collaboration for Inference

4.1 Architectures for Edge-Cloud Collaborative Inference

The increasing demand for low-latency and computationally efficient large language model (LLM) inference has led to the development of various edge-cloud collaborative architectures [12]. These architectures aim to leverage the strengths of both edge and cloud computing to balance latency and computational resources. One such approach is fog computing, which enables low-latency processing and reduces communication overhead by bringing computation closer to the user.

Another approach is hierarchical edge-cloud models, which allow for layered inference and flexible allocation of computational resources. These models enable adaptive handling of varying workloads and can be optimized for specific tasks, such as language translation and sentiment analysis [100].

Recent studies have also explored the use of edge-cloud hybrid architectures for dynamic task allocation, facilitating efficient use of resources and minimizing latency [26]. These architectures enable seamless collaboration between edge and cloud computing, allowing for real-time processing and reducing communication overhead [101].

In addition to these architectures, various optimization techniques have been proposed to enhance the performance of edge-cloud collaborative inference [70]. These techniques include model pruning, quantization, and knowledge distillation, which can reduce the computational footprint of LLMs and improve their efficiency.

The use of edge-cloud collaborative architectures and optimization techniques has significant implications for the deployment of LLMs in resource-constrained environments [5]. However, further research is needed to address the challenges associated with edge-cloud collaborative inference, such as task partitioning, communication protocols, and security considerations [1].

In conclusion, edge-cloud collaborative architectures and optimization techniques have the potential to revolutionize the deployment of LLMs in resource-constrained environments [6]. As the demand for low-latency and computationally efficient LLM inference continues to grow, the development of edge-cloud collaborative architectures and optimization techniques will play a critical role in enabling the widespread adoption of LLMs in various applications [8].

4.2 Task Partitioning and Offloading Strategies

Task partitioning and offloading strategies are crucial components of edge-cloud collaborative inference, as they enable the optimal utilization of resources across the edge and cloud. The primary goal of these strategies is to minimize latency and maximize computational efficiency by determining which parts of the model should be executed on the edge and which on the cloud [26]. Effective partitioning and offloading techniques must consider factors such as computational complexity, memory requirements, and communication overhead to achieve a balance between

local processing and cloud offloading [32]. This balance is essential for achieving low-latency and computationally efficient large language model inference, which is critical for various applications that require real-time processing.

One approach to task partitioning is to divide the model into layers or modules and assign them to either the edge or the cloud based on their computational requirements and memory usage [27]. For example, the early layers of a large language model, which typically require less computation and memory, can be executed on the edge, while the later layers, which are more computationally intensive, can be offloaded to the cloud [54]. This approach can help reduce the communication overhead and latency associated with offloading the entire model to the cloud. Furthermore, techniques such as model pruning and quantization can be used to reduce the computational requirements and memory usage of the model, making it more suitable for edge execution [73].

In addition to static partitioning, dynamic partitioning approaches can be used to adapt to changing resource availability and input data characteristics [95]. This approach can help ensure optimal performance and efficiency by dynamically allocating tasks between the edge and cloud. Offloading strategies can also be designed to minimize latency and maximize throughput, such as offloading the model to the cloud only when the edge device is under heavy load or when the input data requires more computational resources than available on the edge [102]. The use of mixture-of-experts (MoE) models is another approach that can be used for task partitioning and offloading [20]. MoE models consist of multiple experts, each of which is a smaller model that is trained to specialize in a specific part of the input data, and can be used to reduce the computational requirements and memory usage of the model.

Several other strategies have been proposed for task partitioning and offloading, including the use of reinforcement learning [95], graph-based methods [77], and hybrid approaches that combine multiple techniques [27]. These strategies can be used to optimize the performance and efficiency of edge-cloud collaborative inference, enabling the deployment of large language models in resource-constrained environments. As the field continues to evolve, the development of more sophisticated task partitioning and offloading strategies will be crucial for minimizing latency and maximizing computational efficiency. The optimization of communication protocols and data management strategies, which will be discussed in the next section, will also play a critical role in ensuring the efficient and reliable operation of edge-cloud collaborative inference systems [3].

4.3 Communication Protocols and Data Management

Efficient communication protocols and data management strategies are crucial for seamless edge-cloud collaboration in large language model inference. As highlighted in [6], the integration of edge computing and cloud computing enables real-time processing, reduces latency, and improves privacy. However, the communication overhead between edge and cloud can significantly impact the overall performance of the system. To mitigate this, [63] proposes a novel strategy that accelerates offloading by taking advantage of innate properties of Mixture-of-Experts (MoE) large language models.

In the context of MoE models, [103] introduces a resource-efficient inference system that strategically utilizes CPU and GPU resources by determining the optimal execution strategy. This approach enables the efficient offloading of experts to CPU memory, reducing GPU memory requirements and minimizing communication overhead. Furthermore, [85] presents a mixed precision expert offloading system that dynamically replaces less critical cache-miss experts with low-precision versions, reducing expert-loading latency while preserving model accuracy.

Data management is another critical aspect of edge-cloud collaboration. [56] proposes a graph state backtracking algorithm with efficient pruning and hashing strategies to minimize memory overhead during inference. This approach enables the optimization of distributed inference latency and memory usage, making it suitable for edge devices with limited resources.

Additionally, [27] introduces a collaborative edge computing framework that partitions the large language model into shards and deploys them on distributed devices, achieving efficient inference and reducing latency.

The importance of adaptive expert scheduling and memory coordination is highlighted in [104]. This approach combines adaptive expert prefetching and cache-aware routing, reducing model stall time and minimizing latency caused by expert swap-ins. Moreover, [89] proposes a lightweight ML-based caching strategy that adaptively combines recency and frequency signals to maximize expert reuse, significantly reducing storage I/O and improving cache hit rates.

In the context of reliability and efficiency, [90] investigates the reliability of sparse MoE models under stochastic decoding, highlighting the importance of instruction tuning in maintaining output stability. Furthermore, [105] explores the efficacy of embedding scaling as a potent dimension for scaling sparsity, achieving a superior Pareto frontier compared to expert scaling in specific regimes.

As the field continues to evolve, emerging trends and challenges in communication protocols and data management for edge-cloud collaboration will require innovative solutions. The development of more efficient and adaptive expert scheduling algorithms, advanced caching strategies, and optimized data management techniques will be crucial in minimizing latency and ensuring reliable inference. As noted in [83], the integration of edge computing and deep learning will play a vital role in enabling real-time processing and improving privacy, driving the need for continued research and innovation in this area. By addressing these challenges and opportunities, researchers and practitioners can unlock the full potential of edge-cloud collaboration for large language model inference, enabling more efficient, reliable, and scalable solutions for a wide range of applications.

4.4 Optimization Techniques for Edge-Cloud Inference

Optimization techniques are essential for enhancing the performance of edge-cloud collaborative inference, as they aim to minimize latency, reduce energy consumption, and optimize resource utilization. Building on the discussion of efficient communication protocols and data management strategies in the previous section, optimization techniques can be applied to further improve the efficiency and effectiveness of edge-cloud collaborative inference. One of the key techniques in this domain is joint optimization of edge and cloud resources, which involves optimizing the allocation of computational resources between the edge and cloud to minimize latency and maximize throughput [32]. This approach can be further enhanced by incorporating energy-aware optimization techniques, such as dynamic voltage and frequency scaling, to reduce energy consumption in edge-cloud collaborative inference [35].

In addition to joint optimization, adaptive optimization is another important technique that involves dynamically adjusting the edge-cloud collaboration based on varying workloads and conditions [36]. This approach can be achieved through the use of machine learning algorithms that predict the optimal resource allocation and task partitioning between the edge and cloud. Furthermore, techniques such as model pruning and quantization can be applied to reduce the computational complexity and memory requirements of large language models, making them more suitable for edge-cloud collaborative inference [74]. The use of mixture-of-experts (MoE) models is also a promising approach for edge-cloud collaborative inference, as they can activate only a subset of the model’s parameters during computation, reducing computational costs [106].

To further optimize the performance of large language models, techniques such as knowledge distillation and model compression can be used to transfer knowledge from large models to smaller ones, reducing the computational requirements and memory footprint of the models [107]. Recent studies have also explored the use of hardware-aware pruning and quantization techniques to optimize the performance of large language models in edge-cloud collaborative inference [80]. These techniques involve pruning and quantizing the model’s weights and activations based on the hardware characteristics of the edge and cloud devices, resulting in significant

reductions in latency and energy consumption. Moreover, the use of cloud-edge collaborative inference frameworks, such as EdgeShard, can facilitate the collaboration between edge and cloud computing for large language model inference, offering a balance between latency and computational resources [27].

As the field of edge-cloud collaborative inference continues to evolve, the development of new optimization techniques and frameworks will be crucial for further improving the efficiency and effectiveness of large language model inference. The optimization techniques discussed in this section can be used in conjunction with the security and privacy mechanisms discussed in the following section to ensure reliable and trustworthy operation of edge-cloud collaborative inference systems [3]. Future research directions may include the exploration of new optimization techniques, such as reinforcement learning and graph-based methods, to further improve the performance of edge-cloud collaborative inference [42].

4.5 Security and Privacy Considerations

Ensuring the security and privacy of data is crucial for reliable and trustworthy operation in edge-cloud collaborative inference. As [6] highlights, the integration of large language models (LLMs) with edge computing presents a promising solution for enhancing privacy and reducing latency. However, this paradigm also introduces new security and privacy challenges that must be addressed. For instance, [50] discusses the potential risks associated with deploying LLMs on edge devices, including the increased attack surface and potential for data breaches.

One of the primary concerns in edge-cloud collaborative inference is the protection of user data during transmission between the edge and cloud. [51] proposes a novel approach to predict activation sparsity in LLMs, which can help reduce the amount of data transmitted and minimize the risk of data breaches. Additionally, [97] presents a framework for privacy-preserving inference on edge devices, leveraging techniques such as differential noise injection and federated fine-tuning to protect sensitive user data.

Another critical aspect of security and privacy in edge-cloud collaborative inference is access control and authentication. [52] emphasizes the importance of implementing robust access control mechanisms to prevent unauthorized access to edge devices and LLMs. Furthermore, [53] highlights the need for secure authentication protocols to ensure that only authorized devices can participate in collaborative inference.

The use of large language models also raises concerns about model inversion attacks, where an attacker attempts to reconstruct the input data from the model’s outputs. [108] demonstrates the severity of this threat and proposes a two-stage method to execute a prompt inversion attack. To mitigate such attacks, [109] suggests implementing aggregation-aware routing interfaces and distributed prompt-aware scheduling to minimize the exposure of sensitive information.

In addition to these challenges, edge-cloud collaborative inference also presents opportunities for enhancing security and privacy through the use of emerging technologies such as homomorphic encryption and secure multi-party computation. [110] explores the potential of homomorphic encryption for secure LLM inference, while [61] proposes a framework for secure multi-party computation in edge-cloud collaborative inference.

As the field of edge-cloud collaborative inference continues to evolve, it is essential to address the security and privacy challenges associated with this paradigm. By leveraging techniques such as predictive activation sparsity, privacy-preserving inference, and secure access control, we can ensure the reliable and trustworthy operation of LLMs in edge-cloud collaborative inference. Moreover, the integration of emerging technologies such as homomorphic encryption and secure multi-party computation can further enhance the security and privacy of this paradigm. As [101] and [27] demonstrate, the potential benefits of edge-cloud collaborative inference are substantial, and addressing the security and privacy challenges associated with this paradigm is crucial for realizing its full potential. Ultimately, the development of secure and privacy-preserving edge-cloud collaborative inference solutions will require continued innovation and research in

this field, as well as collaboration between academia, industry, and government to establish standards and best practices for secure and trustworthy operation.

4.6 Applications and Use Cases

The applications and use cases of edge-cloud collaborative inference are diverse and rapidly expanding, driven by the need for efficient, low-latency, and privacy-preserving processing of large language models (LLMs) [111]. As discussed in the previous section, ensuring the security and privacy of data is crucial for reliable and trustworthy operation in edge-cloud collaborative inference. Building on this foundation, edge-cloud collaborative inference enables the deployment of LLMs in various scenarios, including smart homes, autonomous vehicles, and healthcare, where real-time processing and privacy are crucial [5]. For instance, in smart homes, edge-cloud collaborative inference can be used for voice-controlled automation systems, enabling seamless interaction between users and smart devices [14].

One of the primary benefits of edge-cloud collaborative inference is its ability to balance latency and computational resources. By partitioning the LLM into smaller components and processing them across edge devices and cloud servers, this approach can significantly reduce latency and improve overall system efficiency [32]. Furthermore, edge-cloud collaborative inference can enhance privacy by processing sensitive data locally on edge devices, reducing the need for data transmission to cloud servers and minimizing the risk of data breaches [50]. This is particularly important in applications where data privacy is a major concern, such as healthcare and finance.

However, edge-cloud collaborative inference also poses several challenges, including the need for efficient task partitioning, communication protocols, and data management strategies [12]. The choice of task partitioning strategy can significantly impact system performance, and finding the optimal partitioning point is crucial for minimizing latency and maximizing throughput [95]. Additionally, communication protocols and data management strategies must be carefully designed to ensure seamless collaboration between edge devices and cloud servers, minimizing communication overhead and ensuring data consistency [60].

Recent studies have explored various approaches to address these challenges, including the use of model pruning, quantization, and knowledge distillation to reduce the computational footprint of LLMs [80]. These techniques can be used to create efficient, smaller models that can be deployed on edge devices, reducing the need for communication with cloud servers and improving overall system efficiency [64]. Moreover, edge-cloud collaborative inference can be combined with other techniques, such as federated learning and transfer learning, to further improve system performance and privacy [96]. As the field continues to evolve, we can expect to see even more innovative applications of edge-cloud collaborative inference, driving further research and development in this area.

The applications of edge-cloud collaborative inference are vast and continue to expand, driven by advances in LLMs, edge computing, and cloud computing. As the demand for efficient, low-latency, and privacy-preserving processing of LLMs continues to grow, edge-cloud collaborative inference is likely to play an increasingly important role in enabling the deployment of these models in various scenarios [62]. Future research directions include the development of more efficient task partitioning strategies, communication protocols, and data management strategies, as well as the exploration of new applications and use cases for edge-cloud collaborative inference [53]. Additionally, the integration of edge-cloud collaborative inference with other techniques, such as federated learning and transfer learning, is likely to be an important area of research in the future [61].

5 Security and Privacy Considerations

5.1 Threat Models and Vulnerability Analysis

The security and privacy of Network Edge Inference for Large Language Models are of paramount importance, as these models handle sensitive user data and operate in resource-constrained environments. Threat models and vulnerability analysis are essential to understanding the potential risks and weaknesses in these systems. One of the primary concerns is data breaches, where unauthorized access to user data can occur due to insecure data transmission or storage [4]. Model inversion attacks are another significant threat, where attackers aim to reconstruct input data from model outputs, compromising user privacy [108].

Edge-specific vulnerabilities, such as limited resources and proximity to user data, also pose significant risks [5]. For instance, the use of edge devices with limited computational capabilities and memory can make them more susceptible to attacks [65]. Moreover, the deployment of Large Language Models in edge environments can exacerbate existing vulnerabilities, such as data breaches and model inversion attacks [1].

To mitigate these risks, various approaches have been proposed, including encryption mechanisms, such as homomorphic encryption and secure multi-party computation [51]. These techniques enable computations on encrypted data, ensuring the confidentiality and integrity of user data. Access control mechanisms, such as differential privacy and federated learning, can also be employed to protect user data and models [3].

Recent studies have also explored the use of secure aggregation techniques to protect the privacy of user contributions during collaborative model updates. Additionally, the development of personalized language models, such as TinyLLM, can help reduce the risk of data breaches and model inversion attacks by enabling on-device inference [54].

Despite these advancements, emerging trends and challenges continue to arise. For instance, the increasing use of edge devices and the proliferation of Large Language Models have created new attack surfaces [6]. Moreover, the complexity of edge environments and the limited resources available can make it challenging to implement robust security measures [27].

In conclusion, the security and privacy of Network Edge Inference for Large Language Models are critical concerns that require careful attention. By understanding the various threat models and vulnerabilities, researchers and practitioners can develop effective mitigation strategies to protect user data and models. Future research directions should focus on developing more robust security mechanisms, such as encryption and access control, and exploring the use of personalized language models and secure aggregation techniques [53]. Ultimately, a comprehensive approach to security and privacy is essential to ensuring the widespread adoption and trustworthiness of Network Edge Inference for Large Language Models [57].

5.2 Encryption and Access Control Mechanisms

The protection of user data is a paramount concern in Network Edge Inference, particularly when dealing with sensitive information such as language models. As discussed earlier, the security and privacy of Network Edge Inference for Large Language Models are of utmost importance, and encryption and access control mechanisms play a crucial role in ensuring the confidentiality and integrity of user data. Homomorphic encryption, for instance, enables computations to be performed on encrypted data, generating an encrypted result that can be decrypted to obtain the desired outcome [4]. This approach allows for secure outsourcing of computations to untrusted environments, making it an attractive solution for edge inference, and can be used in conjunction with other techniques, such as differential privacy and federated learning, to further enhance user privacy.

Secure multi-party computation (SMC) is another technique used to protect user data in Network Edge Inference. SMC enables multiple parties to jointly perform computations on private

data without revealing their individual inputs. However, SMC can be computationally expensive and may require significant communication overhead, making it challenging to implement in resource-constrained edge environments. To address these challenges, researchers have explored the use of emerging technologies, such as blockchain and federated learning, to enhance the security and privacy of Network Edge Inference [26]. Blockchain-based approaches can provide a secure and transparent way to manage access control and data sharing, while federated learning enables multiple devices to collaboratively train models without sharing raw data.

Access control mechanisms are also essential in Network Edge Inference to regulate who can access and manipulate model outputs and user data. Techniques such as role-based access control (RBAC) and attribute-based access control (ABAC) can be used to enforce fine-grained access control policies. These mechanisms can be used to complement encryption and SMC protocols, providing an additional layer of protection for user data. Furthermore, the development of personalized language models, such as TinyLLM, can help reduce the risk of data breaches and model inversion attacks by enabling on-device inference [54].

Despite the advancements in encryption and access control mechanisms, there are still significant challenges to be addressed in Network Edge Inference. The trade-off between security, performance, and energy efficiency is a major concern, as implementing robust security measures can increase computational overhead and energy consumption [76]. Therefore, it is essential to develop and implement efficient security mechanisms that balance security, performance, and energy efficiency. The use of privacy-preserving inference techniques, such as differential privacy and federated learning, can help mitigate these challenges and provide a secure and efficient framework for Network Edge Inference.

In conclusion, encryption and access control mechanisms are essential components of Network Edge Inference, providing a crucial layer of protection for user data. Emerging technologies, such as blockchain and federated learning, offer promising solutions for enhancing the security and privacy of edge inference. As the field continues to evolve, it is essential to prioritize user privacy and develop innovative solutions that balance privacy and performance in Network Edge Inference, ultimately enabling the widespread adoption of these models in various applications, and paving the way for the development of privacy-preserving inference techniques, such as those discussed in the following section.

5.3 Privacy-Preserving Inference Techniques

The increasing use of Large Language Models (LLMs) at the network edge has raised significant concerns regarding privacy and data protection. As these models process vast amounts of sensitive information, it is essential to develop and implement privacy-preserving inference techniques to safeguard user data. This subsection delves into the various approaches designed to protect sensitive information during the inference process, including differential privacy and federated learning.

Differential privacy, a technique introduced in [112], has emerged as a promising approach for protecting user data in LLMs. By adding controlled noise to the model’s outputs, differential privacy ensures that individual data points cannot be identified or linked to specific users.

Federated learning, on the other hand, enables multiple devices to collaboratively train a model without sharing their local data, as discussed in [21] and [30]. This approach has gained significant attention in recent years due to its potential to enhance user privacy while maintaining model performance.

Another promising approach is the use of secure multi-party computation (SMC) protocols, which enable multiple parties to jointly perform computations on private data without revealing their inputs, as discussed in [113].

Recent studies have also explored the use of homomorphic encryption (HE) in LLMs, which enables computations to be performed directly on encrypted data, as discussed in [84].

In addition to these approaches, researchers have also explored the use of privacy-preserving techniques such as data anonymization, data perturbation, and secure data aggregation, as discussed in [83].

In conclusion, privacy-preserving inference techniques are essential for protecting sensitive information in LLMs at the network edge. Differential privacy, federated learning, SMC protocols, HE, and other privacy-preserving techniques have shown significant potential in enhancing user privacy while maintaining model performance. However, these approaches often come with trade-offs, such as reduced model accuracy, increased computational overhead, or communication overhead. Further research is needed to develop and refine these techniques, ensuring that they can be effectively applied in real-world LLM applications, as highlighted in [5]. As the field continues to evolve, it is essential to prioritize user privacy and develop innovative solutions that balance privacy and performance in LLMs.

5.4 Edge-Cloud Collaboration for Secure Inference

The collaboration between edge and cloud computing for secure Large Language Model (LLM) inference has become a crucial aspect of ensuring the efficient and reliable deployment of these models in various applications [32]. By leveraging the strengths of both edge and cloud computing, this collaborative framework can facilitate the secure and efficient inference of LLMs, even in resource-constrained environments [12]. This approach is particularly important in the context of privacy-preserving inference techniques, such as differential privacy and federated learning, which can be used in conjunction with edge-cloud collaboration to enhance the security and privacy of LLM inference.

One of the primary advantages of edge-cloud collaboration for secure LLM inference is the ability to reduce latency while maintaining security [35]. Furthermore, edge-cloud collaboration can enable the use of more secure communication protocols, such as homomorphic encryption and secure multi-party computation, to protect data during transmission [107]. This is especially relevant in the context of emerging trends and technologies, such as Edge AI, which promises to revolutionize the inference capabilities of Large Language Models by enabling more efficient and secure deployment across edge and cloud computing resources.

In addition to security, edge-cloud collaboration can also provide significant improvements in computational efficiency [77]. Moreover, the use of model pruning and quantization techniques can further reduce the computational requirements of LLMs, enabling more efficient inference on edge devices [40]. For example, the application of model pruning can reduce the number of parameters in an LLM, resulting in significant improvements in inference speed and computational efficiency [41]. This, in turn, can facilitate the widespread adoption of LLMs in various applications, including those that require low-latency and high-security inference.

Despite the advantages of edge-cloud collaboration for secure LLM inference, there are also several challenges and limitations that need to be addressed [34]. Furthermore, the use of edge-cloud collaboration requires careful consideration of the trade-offs between latency, computational resources, and security [27]. To address these challenges, researchers have proposed various approaches, including the application of model compression techniques, such as pruning and quantization, to reduce the computational requirements of LLMs and enable more efficient inference on edge devices [25].

In recent years, several approaches have been proposed to address the challenges and limitations of edge-cloud collaboration for secure LLM inference [114]. Another approach is the use of knowledge distillation and transfer learning, which can enable the development of more efficient and secure LLMs, by transferring knowledge from larger models to smaller ones [115]. These approaches can be used in conjunction with edge-cloud collaboration to provide a comprehensive framework for secure and efficient LLM inference.

In conclusion, edge-cloud collaboration for secure LLM inference has the potential to provide significant improvements in latency, computational efficiency, and security [46]. By leveraging

the strengths of both edge and cloud computing, and addressing the challenges and limitations of this approach, it is possible to develop more efficient, secure, and reliable LLM inference mechanisms [39]. As the field continues to evolve, it is essential to prioritize security, privacy, and transparency in edge-cloud collaboration for LLM inference, and to develop innovative solutions that balance these competing demands, ultimately enabling the widespread adoption of LLMs in various applications.

5.5 Emerging Trends and Future Directions

The realm of Network Edge Inference for Large Language Models is witnessing a paradigm shift with the emergence of novel trends and technologies. As the field continues to evolve, it is essential to explore these developments and their implications on security and privacy. One of the key emerging trends is the integration of Edge AI [6], which promises to revolutionize the inference capabilities of Large Language Models. However, this convergence also introduces new security challenges, such as the potential for quantum computers to break certain classical encryption algorithms [67].

Another significant trend is the increasing adoption of Collaborative Edge-Cloud Inference [12], which enables the efficient deployment of Large Language Models across edge and cloud computing resources. This approach not only enhances inference performance but also raises concerns about data privacy and security [52]. To address these concerns, researchers are exploring techniques such as Differential Privacy [97] and Secure Multi-Party Computation [51].

The proliferation of Edge Devices and the Internet of Things (IoT) has also led to an increased focus on Edge-Centric Security [50] and Privacy-Preserving Inference [97]. Techniques such as Federated Learning [96] and Homomorphic Encryption [109] are being explored to protect user data and ensure secure inference.

Furthermore, the development of Explainable AI (XAI) and Transparency in Edge Inference [65] is becoming increasingly important. As Large Language Models are deployed in critical applications, it is essential to understand their decision-making processes and ensure that they are fair, accountable, and transparent [116]. Researchers are working on developing techniques to provide insights into the models' behavior and decisions, which will be crucial for building trust in Edge Inference systems.

In addition to these trends, the evolution of Threat Landscapes [108] is a significant concern. As Edge Inference systems become more prevalent, they will become attractive targets for malicious actors. It is essential to develop robust security measures to protect against these threats and ensure the integrity of Edge Inference systems [15].

In conclusion, the future of Network Edge Inference for Large Language Models will be shaped by the interplay of emerging trends, technological advancements, and evolving threat landscapes. As researchers, it is essential to stay at the forefront of these developments and address the associated security and privacy challenges [55]. By doing so, we can unlock the full potential of Edge Inference and create a more secure, efficient, and transparent AI ecosystem [101]. The integration of new technologies, such as Edge AI [27], will be crucial in shaping the future of Edge Inference. As we move forward, it is essential to prioritize security, privacy, and transparency to ensure that Edge Inference systems are trustworthy and beneficial for society [61].

6 Applications and Use Cases

6.1 Edge Computing Applications for Large Language Models

The integration of Large Language Models (LLMs) with edge computing has opened up new avenues for real-time processing, reduced latency, and improved privacy in various applications. Edge computing enables the deployment of LLMs on edge devices, such as smart home devices,

autonomous vehicles, and healthcare devices, allowing for faster and more secure processing of data [5]. This subsection explores the various applications of LLMs in edge computing scenarios, including smart homes, autonomous vehicles, and healthcare devices.

In smart homes, LLMs can be used for voice-controlled home automation systems, enabling seamless interaction between users and smart devices [14]. For instance, users can control lighting, temperature, and security systems using voice commands, making it easier to manage their homes. Moreover, LLMs can be used for natural language-based navigation in autonomous vehicles, improving the overall driving experience and safety [117]. Autonomous vehicles can use LLMs to understand voice commands, navigate through roads, and make decisions in real-time.

In healthcare, LLMs can be integrated with healthcare devices, such as medical chatbots, to provide personalized patient care and support [6]. Medical chatbots can use LLMs to understand patient queries, provide medical information, and offer personalized advice. Furthermore, LLMs can be used for medical diagnosis, patient data analysis, and clinical decision-making, enabling healthcare professionals to make more accurate and informed decisions.

The use of LLMs in edge computing scenarios also raises several challenges, including model compression, quantization, and knowledge distillation [70]. Model compression techniques, such as pruning and quantization, can be used to reduce the size of LLMs, making them more suitable for deployment on edge devices [3]. Knowledge distillation can also be used to transfer knowledge from large LLMs to smaller models, enabling more efficient deployment on edge devices.

Recent studies have also explored the use of LLMs in edge computing for natural language processing tasks, such as language translation, sentiment analysis, and text summarization [1]. For instance, [26] proposed a personalized inference scheduling framework for diverse LLM services, which can optimize service scheduling and resource allocation solutions within the edge-cloud infrastructure. Moreover, [27] proposed a collaborative edge computing framework for efficient LLM inference, which can achieve up to 50% latency reduction and 2x throughput improvement over baseline methods.

In addition, [53] conducted a comprehensive evaluation of generative LLM inference on representative CPU-based and GPU-accelerated edge devices, measuring key performance indicators, including memory usage, inference speed, and energy consumption. The study found that quantization helps mitigate memory overhead but does not fully eliminate resource bottlenecks, especially for larger models.

In conclusion, the application of LLMs in edge computing scenarios has the potential to revolutionize various industries, including smart homes, autonomous vehicles, and healthcare. However, it also raises several challenges, including model compression, quantization, and knowledge distillation. Recent studies have explored the use of LLMs in edge computing for natural language processing tasks and proposed various frameworks and techniques to optimize service scheduling, resource allocation, and inference efficiency. As the field continues to evolve, it is essential to address these challenges and explore new applications and techniques for LLMs in edge computing [65]. Future research directions include the development of more efficient model compression techniques, the exploration of new applications for LLMs in edge computing, and the investigation of the potential of LLMs in enabling new services and applications [8].

6.2 Natural Language Processing Tasks at the Edge

The increasing demand for real-time natural language processing (NLP) capabilities in edge computing scenarios has led to a growing interest in deploying large language models (LLMs) at the network edge. As discussed in the previous section, the integration of LLMs with edge computing has opened up new avenues for real-time processing, reduced latency, and improved privacy in various applications. Building on this concept, this subsection delves into the use cases for Network Edge Inference in NLP tasks, including language translation, sentiment analysis, and text summarization, highlighting the benefits of performing these tasks at the edge, such as improved latency and reduced bandwidth usage. As [91] suggests, model pruning techniques can

be applied to reduce the computational footprint of LLMs, making them more suitable for edge deployment.

Language translation is a critical NLP task that can significantly benefit from edge inference. By translating text in real-time at the edge, devices can reduce the latency associated with sending data to the cloud for processing, as noted in [14]. This is particularly important for applications where timely communication is crucial, such as in emergency response situations or in environments with limited network connectivity. Furthermore, edge-based translation can enhance user privacy by minimizing the amount of sensitive data transmitted to the cloud. [4] provides insights into the energy costs associated with LLM inference, emphasizing the need for efficient models that can operate within the power constraints of edge devices.

Sentiment analysis, another key NLP task, involves determining the emotional tone or attitude conveyed by a piece of text. Performing sentiment analysis at the edge enables immediate feedback, which is invaluable in applications such as customer service chatbots or social media monitoring tools, as discussed in [53]. The ability to analyze user sentiment in real-time can lead to more responsive and personalized interactions, improving overall user experience. Moreover, edge-based sentiment analysis can help in filtering out inappropriate content or detecting early signs of cyberbullying, contributing to a safer online environment. [74] highlights the redundancy in some layers of LLMs, suggesting that careful model design and pruning can lead to more efficient edge deployment without significant performance degradation.

Text summarization, the process of distilling large documents into concise, informative summaries, is also a task that can benefit from edge inference. By summarizing texts at the edge, devices can quickly provide users with relevant information, facilitating faster decision-making and information retrieval, as [54] demonstrates. This capability is particularly useful in scenarios where users need to rapidly digest large amounts of data, such as in legal or medical professions. Edge-based text summarization can also enhance privacy by reducing the need to transmit sensitive documents to the cloud for processing. [77] discusses a straightforward yet effective pruning method for LLMs, which can be applied to reduce model size and facilitate edge deployment.

The use of Mixture-of-Experts (MoE) models, as discussed in [20] and [21], offers another approach to efficient edge inference. MoE models activate only a subset of the model’s parameters during computation, reducing computational overhead and making them more amenable to edge deployment. This architecture can be particularly beneficial for tasks that require specialized knowledge or handling of diverse input types. [23] presents an open-source MoE model, providing a valuable resource for researchers and developers aiming to explore the potential of MoE architectures in edge computing scenarios.

In conclusion, deploying NLP tasks such as language translation, sentiment analysis, and text summarization at the network edge offers several advantages, including improved latency, enhanced privacy, and reduced bandwidth usage. Techniques such as model pruning, quantization, and the use of MoE models can facilitate the efficient deployment of large language models on edge devices. As the field continues to evolve, it is essential to address the challenges associated with edge inference, including model compression, energy efficiency, and collaborative processing. The future of NLP at the edge looks promising, with potential applications in smart homes, autonomous vehicles, and wearable devices, among others. As [81] suggests, the integration of lightweight LLMs with edge computing can pave the way for AI-native networks, offering ubiquitous intelligence and seamless user experiences. This concept will be further explored in the following section, which will discuss the collaboration between edge and cloud computing for Large Language Model (LLM) inference, and its potential to revolutionize the way we interact with LLMs.

6.3 Enabling New Services and Applications with Edge Inference

The advent of Network Edge Inference has opened up new avenues for innovation in the realm of Large Language Models (LLMs), enabling the development of novel services and applications that can leverage the unique benefits of edge computing. One of the most significant advantages of edge inference is its ability to reduce latency and improve real-time processing capabilities, making it an attractive solution for applications that require instant feedback and response [22]. For instance, edge-based virtual assistants can utilize Network Edge Inference to provide faster and more accurate responses to user queries, enhancing the overall user experience [88].

Another area where Network Edge Inference is poised to make a significant impact is in the realm of augmented reality (AR) experiences. By leveraging the capabilities of edge computing, AR applications can provide more immersive and interactive experiences, with reduced latency and improved responsiveness [60]. For example, an AR application can use edge inference to quickly process user input and provide real-time feedback, creating a more seamless and engaging experience [56]. Moreover, the use of Mixture-of-Experts (MoE) models, as seen in [23], can further enhance the efficiency and effectiveness of edge inference in AR applications.

The potential of Network Edge Inference extends beyond these examples, with applications in areas such as smart homes, autonomous vehicles, and healthcare. In smart homes, edge inference can be used to enable voice-controlled automation systems, allowing users to control various devices and appliances with ease [5]. In autonomous vehicles, edge inference can be utilized to improve navigation and decision-making, enabling vehicles to respond quickly and accurately to changing environments [82]. In healthcare, edge inference can be applied to medical devices, enabling them to provide personalized patient care and support [6].

Despite the promising potential of Network Edge Inference, there are still several challenges that need to be addressed. One of the primary concerns is the issue of model pruning and optimization, which is critical for reducing the computational footprint of LLMs and enabling efficient edge inference [118]. Another challenge is the need for effective expert routing and management, which is essential for ensuring that the right experts are activated at the right time [119]. Furthermore, the development of novel architectures and algorithms that can efficiently leverage the benefits of edge computing is crucial for unlocking the full potential of Network Edge Inference [31].

In conclusion, Network Edge Inference has the potential to revolutionize the way we interact with LLMs, enabling the development of novel services and applications that can leverage the unique benefits of edge computing. By addressing the challenges associated with model pruning, expert routing, and architecture development, we can unlock the full potential of Network Edge Inference and create more efficient, effective, and responsive LLMs. As research in this area continues to evolve, we can expect to see significant advancements in the field, enabling the widespread adoption of Network Edge Inference in various applications and use cases [87]. Ultimately, the future of LLMs lies at the edge, and it is up to us to harness the potential of Network Edge Inference to create a new generation of intelligent, responsive, and interactive applications [120].

6.4 Collaborative Edge-Cloud Inference for Large Language Models

The collaboration between edge and cloud computing for Large Language Model (LLM) inference has emerged as a crucial approach to balance latency and computational resources, building on the benefits of Network Edge Inference discussed in the previous section. This collaborative edge-cloud inference paradigm enables the execution of LLMs on both edge devices and cloud servers, leveraging the strengths of each to optimize performance and efficiency [32]. By partitioning the model between edge and cloud, the computational workload can be distributed, reducing the latency associated with cloud-only inference while minimizing the computational burden on edge devices [12].

One of the primary benefits of edge-cloud collaborative inference is its ability to reduce latency, which is essential for real-time applications that require instant feedback and response. By executing the initial layers of the LLM on the edge device, the latency associated with data transmission to the cloud can be mitigated [27]. Furthermore, this approach enables real-time processing, making it suitable for applications that require immediate responses, such as voice assistants or real-time language translation [107]. However, the collaboration between edge and cloud also introduces new challenges, including the need for efficient communication protocols and data management strategies to ensure seamless collaboration [34].

To address these challenges, several approaches have been proposed to optimize edge-cloud collaborative inference for LLMs. For instance, [77] presents a pruning approach that can be applied to both edge and cloud models, reducing the computational workload while maintaining performance. Additionally, [41] introduces a neural architecture search method that can be used to optimize the model architecture for edge-cloud collaborative inference. Other studies, such as [42], have explored the use of distillation-based methods to optimize LLMs for edge-cloud collaborative inference. These optimization methods can help mitigate the challenges associated with edge-cloud collaborative inference, enabling the deployment of LLMs on edge devices while reducing latency and improving real-time processing capabilities.

The benefits of edge-cloud collaborative inference for LLMs are numerous, and this approach has the potential to revolutionize the way we interact with LLMs. By leveraging the strengths of both edge and cloud computing, this paradigm can enable the deployment of LLMs on edge devices, reducing latency and improving real-time processing capabilities [74]. Moreover, this approach can reduce the computational workload on cloud servers, making it more efficient and cost-effective [76]. As the field continues to evolve, it is likely that edge-cloud collaborative inference will play an increasingly important role in the deployment of LLMs, and its applications will be further explored in the subsequent section.

In conclusion, edge-cloud collaborative inference for LLMs offers a promising approach to balance latency and computational resources, and its benefits and challenges have been extensively studied. By leveraging the strengths of both edge and cloud computing, this paradigm can enable the deployment of LLMs on edge devices, reducing latency and improving real-time processing capabilities. While there are challenges associated with this approach, numerous studies have proposed optimization methods to mitigate these challenges. As research in this area continues to advance, we can expect to see the development of more efficient communication protocols, data management strategies, and optimization methods, ultimately paving the way for innovative applications and use cases [40] [45].

6.5 Real-World Deployment and Case Studies

The deployment of Network Edge Inference for Large Language Models in real-world scenarios has garnered significant attention due to its potential to enhance privacy, reduce latency, and improve overall user experience. Several case studies and deployment scenarios have demonstrated the feasibility and benefits of edge inference, including [27], which leverages collaborative edge computing to facilitate the collaboration among edge devices and cloud servers for jointly performing efficient LLM inference. This approach has shown promising results in reducing latency and improving throughput.

Another notable example is [101], which presents a compute- and memory-efficient tensor parallel inference system to serve 70B-scale models on low-resource edge devices. This system keeps sensitive raw data local in the users' devices and introduces a sliding window memory scheduler to dynamically manage layer weights during inference. The results demonstrate that TPI-LLM can achieve over 80% less time-to-first-token and token latency compared to existing methods.

The [61] framework is also worth mentioning, as it proposes a unified and efficient approach for LLM inference serving, consisting of three main components: resource profiler, batch scheduler,

and LLM deployer. UELLM minimizes resource overhead, reduces inference latency, and lowers SLO violation rates. Compared with state-of-the-art techniques, UELLM reduces the inference latency by 72.3% to 90.3%, enhances GPU utilization by 1.2X to 4.1X, and increases throughput by 1.92X to 4.98X.

In addition to these examples, [121] designs a collaborative inference architecture between a server and its clients to alleviate the throughput limit. This design considers the available resources on both sides and develops a dynamic programming-based algorithm to optimally allocate computation between the server and the client device. The results show that SplitLLM can efficiently distribute the workload, allowing for roughly 1/3 reduction in the server workload, while achieving 19% improvement over a greedy method.

Furthermore, [55] provides an overview of different LLM architectures and introduces several proposed approaches aimed at addressing the issues related to edge deployment, including algorithm-level techniques, hardware acceleration optimization, and edge-cloud collaboration. This study offers insights into current research status, difficulties, and potential opportunities in the related field.

The [53] study presents a comprehensive evaluation of generative LM inference on representative CPU-based and GPU-accelerated edge devices. The results quantify the memory and energy constraints that must be considered for practical real-world deployments, offering concrete insights into the trade-offs between model size, inference performance, and efficiency.

Moreover, [116] proposes a novel collaborative decoding inference system that allows small models to perform on-device inference while selectively consulting a cloud-based large model for critical token generation. This system achieves a 60% performance gain on CommonsenseQA using only a 0.5B model on an M1 MacBook, with under 7% of tokens generation uploaded to the large model in the cloud.

Lastly, [52] conducts a comparative analysis of text-based bias across language model deployments on edge, cloud, and desktop environments. The results demonstrate that Llama-2 running on Raspberry Pi 4 is 43.23% and 21.89% more prone to showing bias over time compared to models running on the desktop and cloud-based environments. The study also proposes the implementation of a feedback loop to correct bias patterns, resulting in a 79.28% reduction in model bias.

In conclusion, the real-world deployment of Network Edge Inference for Large Language Models has shown promising results in various case studies and scenarios. These deployments have demonstrated the potential to reduce latency, improve privacy, and enhance overall user experience. However, they also present challenges and trade-offs that need to be carefully considered, such as model size, inference performance, and energy efficiency. As the field continues to evolve, it is essential to address these challenges and explore new approaches to optimize edge inference for large language models. Future research directions may include the development of more efficient models, improved edge-cloud collaboration architectures, and the integration of edge inference with emerging technologies like 5G and 6G networks.

6.6 Future Directions and Emerging Trends

The integration of Large Language Models (LLMs) with edge computing has paved the way for numerous applications and use cases, transforming the way we interact with and utilize artificial intelligence. As we look to the future, several emerging trends and directions hold significant promise for advancing the field of Network Edge Inference for LLMs. One such trend is the increasing focus on explainability and transparency in LLMs, as highlighted in [60], which is crucial for building trust and accountability in edge inference applications. This is particularly important as LLMs are being deployed in various real-world scenarios, as discussed in the previous section, where their potential to enhance privacy, reduce latency, and improve overall user experience has been demonstrated.

Another key area of research is the development of more efficient and scalable models, such as those proposed in [101], which can enable the deployment of LLMs on resource-constrained edge devices, expanding their reach and accessibility. Furthermore, the use of collaborative edge computing, as discussed in [27], can facilitate the joint execution of LLMs across multiple devices, improving inference efficiency and reducing latency. These advancements have the potential to overcome the challenges associated with edge inference, such as model size, inference performance, and energy efficiency, which were identified as crucial factors in the previous section.

The application of LLMs in edge-based IoT networks, as explored in [122], is another promising area of research. By leveraging LLMs for semantic communication, we can enable more efficient and effective data transmission in IoT networks, paving the way for innovative applications and services. Additionally, the development of privacy-aware routing frameworks, such as [68], can help mitigate privacy concerns in cloud-edge inference, ensuring the secure and trustworthy deployment of LLMs. These developments will be essential in supporting the growing demand for edge-based applications and services, which will be discussed in the following section.

In terms of technical advancements, the use of quantization techniques, as discussed in [123], can significantly reduce the memory footprint of LLMs, making them more suitable for deployment on edge devices. Moreover, the development of novel inference algorithms, such as those proposed in [124], can improve the efficiency and accuracy of LLM inference, enabling real-time processing and decision-making. These technical advancements will play a critical role in unlocking the full potential of LLMs in edge computing applications, which will be explored in more detail in the subsequent sections.

As we move forward, it is essential to address the challenges associated with the deployment of LLMs on edge devices, including energy efficiency, latency, and security. The development of comprehensive carbon footprint estimators, such as [125], can help identify areas for improvement and optimize the environmental sustainability of LLM deployments. Furthermore, the exploration of new architectures and technologies, such as those discussed in [62], can provide valuable insights into the future of LLM inference and its potential applications. By addressing these challenges and continuing to advance the field, we can unlock the full potential of LLMs in edge computing and enable a wide range of innovative applications and use cases.

In conclusion, the future of Network Edge Inference for Large Language Models holds tremendous promise, with numerous emerging trends and directions offering opportunities for innovation and advancement. By building on the foundations established in the previous sections and addressing the challenges and limitations associated with LLM deployment on edge devices, we can enable a wide range of applications and use cases, from smart homes and autonomous vehicles to healthcare and education. As researchers and practitioners, it is essential to continue exploring and developing new techniques, architectures, and technologies that can support the efficient, scalable, and secure deployment of LLMs on edge devices, ultimately transforming the way we interact with and utilize artificial intelligence, as noted in [53].

7 Conclusion

The field of Network Edge Inference for Large Language Models has experienced rapid growth and transformation in recent years, driven by the increasing demand for real-time processing, reduced latency, and improved privacy. As evidenced by [1], the development of large language models has led to significant advancements in natural language processing tasks, including language translation, question answering, and text generation. However, the deployment of these models on edge devices poses substantial challenges due to their computational intensity and memory requirements.

To address these challenges, various approaches have been proposed, including model compression techniques such as quantization, pruning, and knowledge distillation, as discussed

in [3]. These techniques aim to reduce the computational footprint of large language models while maintaining their performance. Additionally, [69] highlights the importance of optimizing training and inference processes for large language models, including the use of distributed training and inference methods.

The concept of edge-cloud collaboration has also gained significant attention, as it offers a promising solution for balancing latency and computational resources. As demonstrated in [12], edge-cloud collaborative inference can reduce end-to-end latency and improve accuracy by leveraging the strengths of both edge and cloud computing. Furthermore, [26] introduces a personalized inference scheduling framework that optimizes service scheduling and resource allocation for diverse LLM services.

Despite the progress made in this field, several challenges and open research questions remain. As noted in [70], the efficient inference of large language models on edge devices is still a topic of ongoing research, with many opportunities for innovation and improvement. Moreover, [6] highlights the need for further research on the development of edge general intelligence, which requires the integration of large language models with other AI technologies.

In terms of future research directions, [126] suggests that the application of large language models to code generation and programming tasks is a promising area of research. Additionally, [127] emphasizes the importance of addressing the challenges associated with large language models, including their computational requirements, memory usage, and potential biases.

As the field of Network Edge Inference for Large Language Models continues to evolve, it is essential to consider the potential applications and implications of these technologies. As discussed in [5], the integration of large language models with edge computing can enable a wide range of applications, including smart homes, autonomous vehicles, and healthcare devices. However, [52] also highlights the need to address potential biases and fairness concerns in edge language models.

In conclusion, the field of Network Edge Inference for Large Language Models is rapidly advancing, with significant progress made in model compression, edge-cloud collaboration, and personalized inference scheduling. However, ongoing research is needed to address the challenges and open questions in this field, including the development of more efficient inference methods, the integration of large language models with other AI technologies, and the mitigation of potential biases and fairness concerns. As noted in [65], the future of large language models on edge devices holds great promise, but it requires careful consideration of the technical, social, and ethical implications of these technologies. By continuing to advance the state-of-the-art in Network Edge Inference for Large Language Models, we can unlock the full potential of these technologies and enable a wide range of innovative applications and services.

References

- [1] *Large Language Models: A Survey*. arXiv:2402.06196, 2024. <https://arxiv.org/abs/2402.06196>
- [2] *Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM*. DOI:10.1145/3458817.3476209, 2021. <https://doi.org/10.1145/3458817.3476209>
- [3] *A Survey on Model Compression and Acceleration for Pretrained Language Models*. DOI:10.1609/aaai.v37i9.26255, 2023. <https://doi.org/10.1609/aaai.v37i9.26255>
- [4] *From Words to Watts: Benchmarking the Energy Costs of Large Language Model Inference*. DOI:10.1109/hpec58863.2023.10363447, 2023. <https://doi.org/10.1109/hpec58863.2023.10363447>
- [5] *Mobile Edge Intelligence for Large Language Models: A Contemporary Survey*. DOI:10.1109/comst.2025.3527641, 2025. <https://doi.org/10.1109/comst.2025.3527641>

- [6] *Toward Edge General Intelligence via Large Language Models: Opportunities and Challenges*. DOI:10.1109/mnet.2025.3539338, 2025. <https://doi.org/10.1109/mnet.2025.3539338>
- [7] *A Review of Current Trends, Techniques, and Challenges in Large Language Models (LLMs)*. DOI:10.3390/app14052074, 2024. <https://doi.org/10.3390/app14052074>
- [8] *Large language models: an overview of foundational architectures, recent trends, and a new taxonomy*. DOI:10.1007/s42452-025-07668-w, 2025. <https://doi.org/10.1007/s42452-025-07668-w>
- [9] *Speed and Conversational Large Language Models: Not All Is About Tokens per Second*. DOI:10.1109/mc.2024.3399384, 2024. <https://doi.org/10.1109/mc.2024.3399384>
- [10] *Achieving Peak Performance for Large Language Models: A Systematic Review*. DOI:10.1109/access.2024.3424945, 2024. <https://doi.org/10.1109/access.2024.3424945>
- [11] *LLMs on a Budget: System-Level Approaches to Power-Efficient and Scalable Fine-Tuning*. DOI:10.1109/ojcs.2025.3580498, 2025. <https://doi.org/10.1109/ojcs.2025.3580498>
- [12] *Cloud-Edge Collaborative Inference with Network Pruning*. DOI:10.3390/electronics12173598, 2023. <https://doi.org/10.3390/electronics12173598>
- [13] *Pipeline Parallelism for Inference on Heterogeneous Edge Computing*. arXiv:2110.14895, 2021. <https://arxiv.org/abs/2110.14895>
- [14] *EdgeBERT: Sentence-Level Energy Optimizations for Latency-Aware Multi-Task NLP Inference*. DOI:10.1145/3466752.3480095, 2021. <https://doi.org/10.1145/3466752.3480095>
- [15] *AdapMoE: Adaptive Sensitivity-Based Expert Gating and Management for Efficient MoE Inference*. DOI:10.1145/3676536.3676741, 2024. <https://doi.org/10.1145/3676536.3676741>
- [16] *Designing Efficient LLM Accelerators for Edge Devices*. arXiv:2408.00462, 2024. <https://arxiv.org/abs/2408.00462>
- [17] *PowerInfer-2: Fast Large Language Model Inference on a Smartphone*. arXiv:2406.06282, 2024. <https://arxiv.org/abs/2406.06282>
- [18] *Lookahead: An Inference Acceleration Framework for Large Language Model with Lossless Generation Accuracy*. DOI:10.1145/3637528.3671614, 2024. <https://doi.org/10.1145/3637528.3671614>
- [19] *MegaScale: Scaling Large Language Model Training to More Than 10, 000 GPUs*. arXiv:2402.15627, 2024. <https://arxiv.org/abs/2402.15627>
- [20] *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*. arXiv:1701.06538, 2017. <https://arxiv.org/abs/1701.06538>
- [21] *Efficient Large Scale Language Modeling with Mixtures of Experts*. DOI:10.18653/v1/2022.emnlp-main.804, 2022. <https://doi.org/10.18653/v1/2022.emnlp-main.804>
- [22] *GLaM: Efficient Scaling of Language Models with Mixture-of-Experts*. arXiv:2112.06905, 2021. <https://arxiv.org/abs/2112.06905>
- [23] *OpenMoE: An Early Effort on Open Mixture-of-Experts Language Models*. arXiv:2402.01739, 2024. <https://arxiv.org/abs/2402.01739>
- [24] *Lory: Fully Differentiable Mixture-of-Experts for Autoregressive Language Model Pre-training*. arXiv:2405.03133, 2024. <https://arxiv.org/abs/2405.03133>

- [25] *Mosaic: Composite Projection Pruning for Resource-efficient LLMs.* DOI:10.1016/j.future.2025.108056, 2025. <https://doi.org/10.1016/j.future.2025.108056>
- [26] *PerLLM: Personalized Inference Scheduling with Edge-Cloud Collaboration for Diverse LLM Services.* arXiv:2405.14636, 2024. <https://arxiv.org/abs/2405.14636>
- [27] *EdgeShard: Efficient LLM Inference via Collaborative Edge Computing.* arXiv:2405.14371, 2024. <https://arxiv.org/abs/2405.14371>
- [28] *BASE Layers: Simplifying Training of Large, Sparse Models.* arXiv:2103.16716, 2021. <https://arxiv.org/abs/2103.16716>
- [29] *Predicting on the Edge: Identifying Where a Larger Model Does Better.* arXiv:2202.07652, 2022. <https://arxiv.org/abs/2202.07652>
- [30] *Mixture of Experts in Large Language Models.* arXiv:2507.11181, 2025. <https://arxiv.org/abs/2507.11181>
- [31] *EPS-MoE: Expert Pipeline Scheduler for Cost-Efficient MoE Inference.* arXiv:2410.12247, 2024. <https://arxiv.org/abs/2410.12247>
- [32] *Optimal Model Placement and Online Model Splitting for Device-Edge Co-Inference.* DOI:10.1109/twc.2022.3165824, 2022. <https://doi.org/10.1109/twc.2022.3165824>
- [33] *A Study of Optimizations for Fine-tuning Large Language Models.* arXiv:2406.02290, 2024. <https://arxiv.org/abs/2406.02290>
- [34] *PyramidInfer: Pyramid KV Cache Compression for High-throughput LLM Inference.* arXiv:2405.12532, 2024. <https://arxiv.org/abs/2405.12532>
- [35] *Measuring and Improving the Energy Efficiency of Large Language Models Inference.* DOI:10.1109/access.2024.3409745, 2024. <https://doi.org/10.1109/access.2024.3409745>
- [36] *Adaptive and Resilient Model-Distributed Inference in Edge Computing Systems.* DOI:10.1109/ojcoms.2023.3280174, 2023. <https://doi.org/10.1109/ojcoms.2023.3280174>
- [37] *SqueezeAttention: 2D Management of KV-Cache in LLM Inference via Layer-wise Optimal Budget.* arXiv:2404.04793, 2024. <https://arxiv.org/abs/2404.04793>
- [38] *Cache Me If You Must: Adaptive Key-Value Quantization for Large Language Models.* arXiv:2501.19392, 2025. <https://arxiv.org/abs/2501.19392>
- [39] *Turbo Sparse: Achieving LLM SOTA Performance with Minimal Activated Parameters.* arXiv:2406.05955, 2024. <https://arxiv.org/abs/2406.05955>
- [40] *FoldGPT: Simple and Effective Large Language Model Compression Scheme.* arXiv:2407.00928, 2024. <https://arxiv.org/abs/2407.00928>
- [41] *LLaMA-NAS: Efficient Neural Architecture Search for Large Language Models.* arXiv:2405.18377, 2024. <https://arxiv.org/abs/2405.18377>
- [42] *Puzzle: Distillation-Based NAS for Inference-Optimized LLMs.* arXiv:2411.19146, 2024. <https://arxiv.org/abs/2411.19146>
- [43] *Inference Optimizations for Large Language Models: Effects, Challenges, and Practical Considerations.* arXiv:2408.03130, 2024. <https://arxiv.org/abs/2408.03130>
- [44] *QEFT: Quantization for Efficient Fine-Tuning of LLMs.* arXiv:2410.08661, 2024. <https://arxiv.org/abs/2410.08661>
- [45] *MoQAE: Mixed-Precision Quantization for Long-Context LLM Inference via Mixture of Quantization-Aware Experts.* arXiv:2506.07533, 2025. <https://arxiv.org/abs/2506.07533>

- [46] *Radio: Rate-Distortion Optimization for Large Language Model Compression.* arXiv:2505.03031, 2025. <https://arxiv.org/abs/2505.03031>
- [47] *Bitnet.cpp: Efficient Edge Inference for Ternary LLMs.* arXiv:2502.11880, 2025. <https://arxiv.org/abs/2502.11880>
- [48] *AWP: Activation-Aware Weight Pruning and Quantization with Projected Gradient Descent.* arXiv:2506.10205, 2025. <https://arxiv.org/abs/2506.10205>
- [49] *SDQ: Sparse Decomposed Quantization for LLM Inference.* arXiv:2406.13868, 2024. <https://arxiv.org/abs/2406.13868>
- [50] *Distributed Threat Intelligence at the Edge Devices: A Large Language Model-Driven Approach.* DOI:10.1109/compsac61105.2024.00206, 2024. <https://doi.org/10.1109/compsac61105.2024.00206>
- [51] *Comet: Accelerating Private Inference for Large Language Model by Predicting Activation Sparsity.* DOI:10.1109/sp61157.2025.00182, 2025. <https://doi.org/10.1109/sp61157.2025.00182>
- [52] *Biases in Edge Language Models: Detection, Analysis, and Mitigation.* arXiv:2502.11349, 2025. <https://arxiv.org/abs/2502.11349>
- [53] *Sometimes Painful but Promising: Feasibility and Trade-Offs of On-Device Language Model Inference.* DOI:10.1145/3788870, 2026. <https://doi.org/10.1145/3788870>
- [54] *TinyLLM: A Framework for Training and Deploying Language Models at the Edge Computers.* arXiv:2412.15304, 2024. <https://arxiv.org/abs/2412.15304>
- [55] *A Comprehensive Analysis on Edge Deployment and Optimization of Llms.* DOI:10.54097/jcb63r28, 2026. <https://doi.org/10.54097/jcb63r28>
- [56] *MemoriaNova: Optimizing Memory-Aware Model Inference for Edge Computing.* DOI:10.1145/3701997, 2024. <https://doi.org/10.1145/3701997>
- [57] *WISP: Waste- and Interference-Suppressed Distributed Speculative LLM Serving at the Edge via Dynamic Drafting and SLO-Aware Batching.* arXiv:2601.11652, 2026. <https://arxiv.org/abs/2601.11652>
- [58] *User Association and Resource Allocation in Large Language Model Based Mobile Edge Computing System over 6G Wireless Communications.* DOI:10.1109/vtc2024-spring62846.2024.10683177, 2024. <https://doi.org/10.1109/vtc2024-spring62846.2024.10683177>
- [59] *Collaborative Edge-to-Server Inference for Vision-Language Models.* arXiv:2512.16349, 2025. <https://arxiv.org/abs/2512.16349>
- [60] *Toward Improving Ensemble-Based Collaborative Inference at the Edge.* DOI:10.1109/access.2024.3351308, 2024. <https://doi.org/10.1109/access.2024.3351308>
- [61] *UELLM: A Unified and Efficient Approach for LLM Inference Serving.* arXiv:2409.14961, 2024. <https://arxiv.org/abs/2409.14961>
- [62] *AI at Scale: The Infrastructure Revolution Enabling GPT-Class Large Language Models.* DOI:10.32996/jcsts.2025.7.4.38, 2025. <https://doi.org/10.32996/jcsts.2025.7.4.38>
- [63] *Mitigating Edge Machine Learning Inference Bottlenecks: An Empirical Study on Accelerating Google Edge Models.* arXiv:2103.00768, 2021. <https://arxiv.org/abs/2103.00768>

- [64] *Quantized Transformer Language Model Implementations on Edge Devices*. DOI:10.1109/icmla58977.2023.00104, 2023. <https://doi.org/10.1109/icmla58977.2023.00104>
- [65] *On-Device Language Models: A Comprehensive Review*. arXiv:2409.00088, 2024. <https://arxiv.org/abs/2409.00088>
- [66] *Sustainable LLM Inference for Edge AI: Evaluating Quantized LLMs for Energy Efficiency, Output Accuracy, and Inference Latency*. DOI:10.1145/3767742, 2025. <https://doi.org/10.1145/3767742>
- [67] *DEFER: Distributed Edge Inference for Deep Neural Networks*. DOI:10.1109/comsnets53615.2022.9668515, 2022. <https://doi.org/10.1109/comsnets53615.2022.9668515>
- [68] *PRISM: Privacy-Aware Routing for Adaptive Cloud-Edge LLM Inference via Semantic Sketch Collaboration*. arXiv:2511.22788, 2025. <https://arxiv.org/abs/2511.22788>
- [69] *Efficient LLMs Training and Inference: An Introduction*. DOI:10.1109/access.2024.3501358, 2024. <https://doi.org/10.1109/access.2024.3501358>
- [70] *A Survey on Efficient Inference for Large Language Models*. arXiv:2404.14294, 2024. <https://arxiv.org/abs/2404.14294>
- [71] *Towards Efficient Generative Large Language Model Serving: A Survey from Algorithms to Systems*. DOI:10.1145/3754448, 2025. <https://doi.org/10.1145/3754448>
- [72] *Structured Pruning of Large Language Models*. DOI:10.18653/v1/2020.emnlp-main.496, 2020. <https://doi.org/10.18653/v1/2020.emnlp-main.496>
- [73] *Efficient Transformer-based Large Scale Language Representations using Hardware-friendly Block Structured Pruning*. DOI:10.18653/v1/2020.findings-emnlp.286, 2020. <https://doi.org/10.18653/v1/2020.findings-emnlp.286>
- [74] *ShortGPT: Layers in Large Language Models are More Redundant Than You Expect*. arXiv:2403.03853, 2024. <https://arxiv.org/abs/2403.03853>
- [75] *SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot*. arXiv:2301.00774, 2023. <https://arxiv.org/abs/2301.00774>
- [76] *SliceGPT: Compress Large Language Models by Deleting Rows and Columns*. arXiv:2401.15024, 2024. <https://arxiv.org/abs/2401.15024>
- [77] *A Simple and Effective Pruning Approach for Large Language Models*. arXiv:2306.11695, 2023. <https://arxiv.org/abs/2306.11695>
- [78] *Shortened LLaMA: A Simple Depth Pruning for Large Language Models*. arXiv:2402.02834, 2024. <https://arxiv.org/abs/2402.02834>
- [79] *APT: Adaptive Pruning and Tuning Pretrained Language Models for Efficient Training and Inference*. arXiv:2401.12200, 2024. <https://arxiv.org/abs/2401.12200>
- [80] *Hape: Hardware-Aware LLM Pruning For Efficient On-Device Inference Optimization*. DOI:10.1145/3744244, 2025. <https://doi.org/10.1145/3744244>
- [81] *Unlocking the Path Towards AI-Native Networks with Optimized Lightweight Large Language Models*. DOI:10.1109/meditcom61057.2024.10621076, 2024. <https://doi.org/10.1109/meditcom61057.2024.10621076>
- [82] *Distributed and Collaborative High-Speed Inference Deep Learning for Mobile Edge with Topological Dependencies*. DOI:10.1109/tcc.2020.2978846, 2020. <https://doi.org/10.1109/tcc.2020.2978846>

- [83] *Enabling Deep Learning for All-in EDGE paradigm.* arXiv:2204.03326, 2022. <https://arxiv.org/abs/2204.03326>
- [84] *EDGE EXCHANGEABLE MODELS FOR INTERACTION NETWORKS.* DOI:10.1080/01621459.2017.1341413, 2017. <https://doi.org/10.1080/01621459.2017.1341413>
- [85] *HOBbit: A Mixed Precision Expert Offloading System for Fast MoE Inference.* arXiv:2411.01433, 2024. <https://arxiv.org/abs/2411.01433>
- [86] *SSD Offloading for LLM Mixture-of-Experts Weights Considered Harmful in Energy Efficiency.* DOI:10.1109/lca.2025.3592563, 2025. <https://doi.org/10.1109/lca.2025.3592563>
- [87] *FLAME-MoE: A Transparent End-to-End Research Platform for Mixture-of-Experts Language Models.* arXiv:2505.20225, 2025. <https://arxiv.org/abs/2505.20225>
- [88] *NetGPT: A Native-AI Network Architecture Beyond Provisioning Personalized Generative Services.* arXiv:2307.06148, 2023. <https://arxiv.org/abs/2307.06148>
- [89] *FlashMoE: Reducing SSD I/O Bottlenecks via ML-Based Cache Replacement for Mixture-of-Experts Inference on Edge Devices.* arXiv:2601.17063, 2026. <https://arxiv.org/abs/2601.17063>
- [90] *Reliability Under Randomness: An Empirical Analysis of Sparse and Dense Language Models Across Decoding Temperatures.* arXiv:2601.00942, 2026. <https://arxiv.org/abs/2601.00942>
- [91] *Efficient Contextualized Representation: Language Model Pruning for Sequence Labeling.* DOI:10.18653/v1/d18-1153, 2018. <https://doi.org/10.18653/v1/d18-1153>
- [92] *OWQ: Outlier-Aware Weight Quantization for Efficient Fine-Tuning and Inference of Large Language Models.* DOI:10.1609/aaai.v38i12.29237, 2024. <https://doi.org/10.1609/aaai.v38i12.29237>
- [93] *Low-Rank Tensor Approximation of Weights in Large Language Models via Cosine Lanczos Bidiagonalization.* arXiv:2601.17112, 2026. <https://arxiv.org/abs/2601.17112>
- [94] *Communication-Efficient Distributed On-Device LLM Inference Over Wireless Networks.* DOI:10.1109/jstsp.2025.3581478, 2025. <https://doi.org/10.1109/jstsp.2025.3581478>
- [95] *Adaptive Layer Splitting for Wireless LLM Inference in Edge Computing: A Model-Based Reinforcement Learning Approach.* arXiv:2406.02616, 2024. <https://arxiv.org/abs/2406.02616>
- [96] *Collaboration of Large Language Models and Small Recommendation Models for Device-Cloud Recommendation.* DOI:10.1145/3690624.3709335, 2025. <https://doi.org/10.1145/3690624.3709335>
- [97] *Efficient TinyML Architectures for On-Device Small Language Models: Privacy-Preserving Inference at the Edge.* DOI:10.56127/ijst.v3i3.1958, 2024. <https://doi.org/10.56127/ijst.v3i3.1958>
- [98] *Fast and Slow Generating: An Empirical Study on Large and Small Language Models Collaborative Decoding.* arXiv:2406.12295, 2024. <https://arxiv.org/abs/2406.12295>
- [99] *Intelligent Orchestration of Distributed Large Foundation Model Inference at the Edge.* DOI:10.37256/cnc.3220256807, 2025. <https://doi.org/10.37256/cnc.3220256807>
- [100] *STI: Turbocharge NLP Inference at the Edge via Elastic Pipelining.* DOI:10.1145/3575693.3575698, 2023. <https://doi.org/10.1145/3575693.3575698>

- [101] *TPI-LLM: Serving 70B-scale LLMs Efficiently on Low-resource Edge Devices.* arXiv:2410.00531, 2024. <https://arxiv.org/abs/2410.00531>
- [102] *Edge-LLM Inference With Cost-Aware Layer Allocation and Adaptive Scheduling.* DOI:10.1109/access.2025.3592308, 2025. <https://doi.org/10.1109/access.2025.3592308>
- [103] *Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models.* arXiv:2402.07033, 2024. <https://arxiv.org/abs/2402.07033>
- [104] *ExpertFlow: Adaptive Expert Scheduling and Memory Coordination for Efficient MoE Inference.* arXiv:2510.26730, 2025. <https://arxiv.org/abs/2510.26730>
- [105] *Scaling Embeddings Outperforms Scaling Experts in Language Models.* arXiv:2601.21204, 2026. <https://arxiv.org/abs/2601.21204>
- [106] *Efficient Expert Pruning for Sparse Mixture-of-Experts Language Models: Enhancing Performance and Reducing Inference Costs.* arXiv:2407.00945, 2024. <https://arxiv.org/abs/2407.00945>
- [107] *QA-LoRA: Quantization-Aware Low-Rank Adaptation of Large Language Models.* arXiv:2309.14717, 2023. <https://arxiv.org/abs/2309.14717>
- [108] *Prompt Inversion Attack Against Collaborative Inference of Large Language Models.* DOI:10.1109/sp61157.2025.00160, 2025. <https://doi.org/10.1109/sp61157.2025.00160>
- [109] *ALTO: An Efficient Network Orchestrator for Compound AI Systems.* DOI:10.1145/3642970.3655844, 2024. <https://doi.org/10.1145/3642970.3655844>
- [110] *Aqua: Network-Accelerated Memory Offloading for LLMs in Scale-Up GPU Domains.* DOI:10.1145/3676641.3715983, 2025. <https://doi.org/10.1145/3676641.3715983>
- [111] *LLMCad: Fast and Scalable On-device Large Language Model Inference.* arXiv:2309.04255, 2023. <https://arxiv.org/abs/2309.04255>
- [112] *Scalable Mutual Information Estimation Using Dependence Graphs.* DOI:10.1109/icassp.2019.8683351, 2019. <https://doi.org/10.1109/icassp.2019.8683351>
- [113] *Relation Learning on Social Networks with Multi-Modal Graph Edge Variational Autoencoders.* DOI:10.1145/3336191.3371829, 2020. <https://doi.org/10.1145/3336191.3371829>
- [114] *FP6-LLM: Efficiently Serving Large Language Models Through FP6-Centric Algorithm-System Co-Design.* arXiv:2401.14112, 2024. <https://arxiv.org/abs/2401.14112>
- [115] *CE-LoRA: Computation-Efficient LoRA Fine-Tuning for Language Models.* arXiv:2502.01378, 2025. <https://arxiv.org/abs/2502.01378>
- [116] *Token Level Routing Inference System for Edge Devices.* arXiv:2504.07878, 2025. <https://arxiv.org/abs/2504.07878>
- [117] *Large Language Models for Networking: Applications, Enabling Techniques, and Challenges.* DOI:10.1109/mnet.2024.3435752, 2024. <https://doi.org/10.1109/mnet.2024.3435752>
- [118] *MoE-Pruner: Pruning Mixture-of-Experts Large Language Model using the Hints from Its Router.* arXiv:2410.12013, 2024. <https://arxiv.org/abs/2410.12013>
- [119] *AdaMoE: Token-Adaptive Routing with Null Experts for Mixture-of-Experts Language Models.* arXiv:2406.13233, 2024. <https://arxiv.org/abs/2406.13233>
- [120] *Pangu Pro MoE: Mixture of Grouped Experts for Efficient Sparsity.* arXiv:2505.21411, 2025. <https://arxiv.org/abs/2505.21411>

- [121] *SplitLLM: Collaborative Inference of LLMs for Model Placement and Throughput Optimization.* arXiv:2410.10759, 2024. <https://arxiv.org/abs/2410.10759>
- [122] *Large Language Models (LLMs) for Semantic Communication in Edge-based IoT Networks.* arXiv:2407.20970, 2024. <https://arxiv.org/abs/2407.20970>
- [123] *On the Compressibility of Quantized Large Language Models.* arXiv:2403.01384, 2024. <https://arxiv.org/abs/2403.01384>
- [124] *Fast Inference for Augmented Large Language Models.* arXiv:2410.18248, 2024. <https://arxiv.org/abs/2410.18248>
- [125] *CO2-Meter: A Comprehensive Carbon Footprint Estimator for LLMs on Edge Devices.* arXiv:2511.08575, 2025. <https://arxiv.org/abs/2511.08575>
- [126] *Large Language Models Meet NL2Code: A Survey.* DOI:10.18653/v1/2023.acl-long.411, 2023. <https://doi.org/10.18653/v1/2023.acl-long.411>
- [127] *Road of Large Language Model: Source, Challenge, and Future Perspectives.* DOI:10.34133/research.0655, 2025. <https://doi.org/10.34133/research.0655>